

Contents

Contents	1
AI Voice Assistant Based on OpenAI Realtime model	2
About the Project.....	2
Cautions	2
About OpenAI	2
OpenAI-Realtime-Embedded Disclaimer	3
ESP32 S3 Hardware Specifications	4
Freenove ESP32-S3 WROOM Board.....	4
Audio Circuit.....	5
Install CH343 Driver (Required).....	6
OpenAI Code	17
Visual Studio Code	17
Windows	17
Mac	20
Linux	21
ESP-IDF V5.4.1	24
OpenAI Code.....	30
Code Downloading	30
Configuring Code Environment	32
Registering Open API Keys.....	36
Code Compilation and Uploading	42

AI Voice Assistant Based on OpenAI Realtime model

This project applies the Media Kit to develop an AI voice assistant for conversations with OpenAI. It requires basic programming skills and some familiarity with OpenAI.

About the Project

This voice assistant project (<https://github.com/Freenove/openai-realtime-embedded>) is based on OpenAI's open-source project [openai-realtime-embedded](https://github.com/openai/openai-realtime-embedded) (<https://github.com/openai/openai-realtime-embedded>). It enables embedded devices to call OpenAI's Realtime Model APIs, such as GPT-4o Realtime and GPT-4o Mini Realtime, allowing you to build your own AI voice assistant.

Freenove has adapted this project for its Media Kit product. This article will guide you on how to run it on the Media Kit.

Cautions

- **Project Copyright:** The original author of this project is OpenAI. Freenove forked and adapted it for Media Kit, and the project is licensed under the MIT License.
- **Supported Countries/Regions:** This project uses OpenAI's GPT-4o RealTime API, which is not available in all countries/regions. Please check OpenAI's supported countries list: <https://platform.openai.com/docs/supported-countries>.
If you cannot access this link, OpenAI's services are likely unavailable in your location.
- **Supported Languages:** OpenAI supports many languages but has no official list. For reference, see: <https://platform.openai.com/docs/api-reference/realtime-sessions/create> (https://en.wikipedia.org/wiki/List_of_ISO_639_language_codes)
- **Pricing:** The GPT-4o RealTime API is a paid service, and you must purchase credits from OpenAI to use it.
- **Seeking Help:** If you have followed the tutorial but still encounter issues, contact us at support@freenove.com
Note: Since both the project and API are provided by OpenAI, if OpenAI discontinues them, we will also remove related documentation, tutorials, and code.

About OpenAI

OpenAI is a leading artificial intelligence research and deployment company committed to ensuring that artificial general intelligence (AGI) benefits all of humanity. Guided by principles of openness, collaboration, and safety, the organization drives the development and practical application of cutting-edge models. These include the renowned GPT series of language models, the DALL-E image generation model, as well as speech recognition and synthesis tools like Whisper.

OpenAI not only provides powerful AI models but also offers API services that enable developers to easily integrate these models into their own products. Recently, OpenAI launched its new Realtime API, which significantly enhances conversational naturalness through **direct streaming of audio input and output**, with automatic interruption handling capabilities.

However, it's important to note that:

1. **OpenAI currently does not offer free services**
2. The Realtime API currently only supports:
GPT-4o and GPT-4o mini models
The latest transcription models: GPT-4o Transcribe and GPT-4o mini Transcribe

For more information about Realtime API, please refer to [Realtime API - OpenAI API](#)

OpenAI-Realtime-Embedded Disclaimer

This implementation is an adaptation of the open-source project available at <https://github.com/openai/openai-realtime-embedded>, provided for third-party learning and AI functionality testing purposes, without any promotion or support for commercial applications. This tutorial is intended solely for personal learning and development by technology enthusiasts.

Notes:

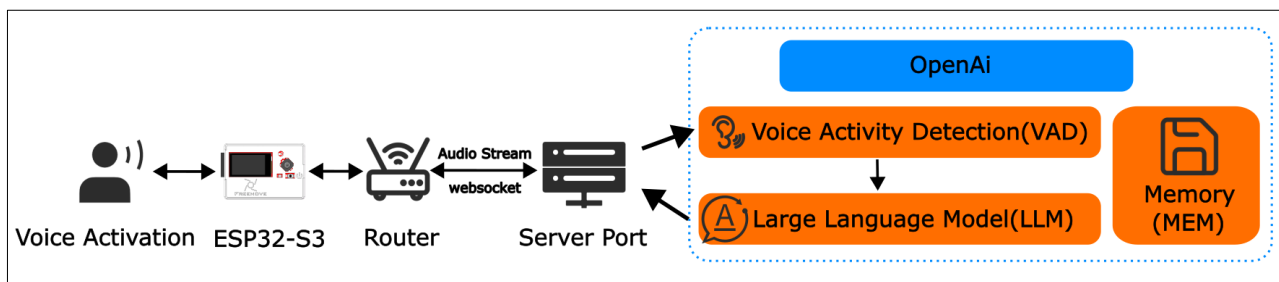
1. As this is a third-party open-source project, if you encounter issues during your learning process, please submit an issue to the original repository <https://github.com/openai/openai-realtime-embedded/issues>
2. The OpenAI API is a paid service. You must subscribe and enable billing to access any API functionality. Without an active paid account, all OpenAI API features will be unavailable.
3. To use OpenAI services, you must apply for your own API key directly from OpenAI. We do not provide or share any API key information.

For details about OpenAI API access, please visit:

<https://platform.openai.com/docs/api-reference/introduction>

Please see the following flowchart.

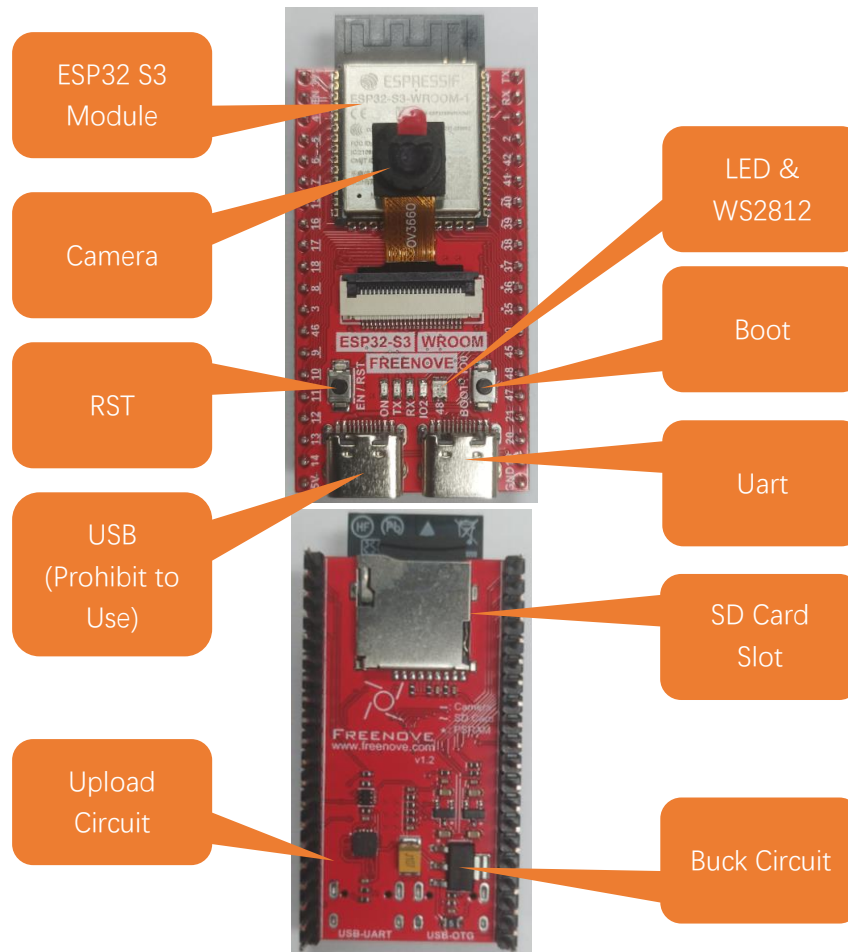
- The ESP32-S3 executes the openai-realtime-embedded source code, establishes a WiFi connection, and streams audio data to OpenAI's servers.
- Upon receiving the audio stream, OpenAI's servers process the data, generate text responses, and synthesize corresponding audio, which is then transmitted back to the ESP32-S3 via WiFi.
- In this project, the ESP32-S3 communicates with OpenAI's servers using the WebSocket protocol.



ESP32 S3 Hardware Specifications

Freenove ESP32-S3 WROOM Board

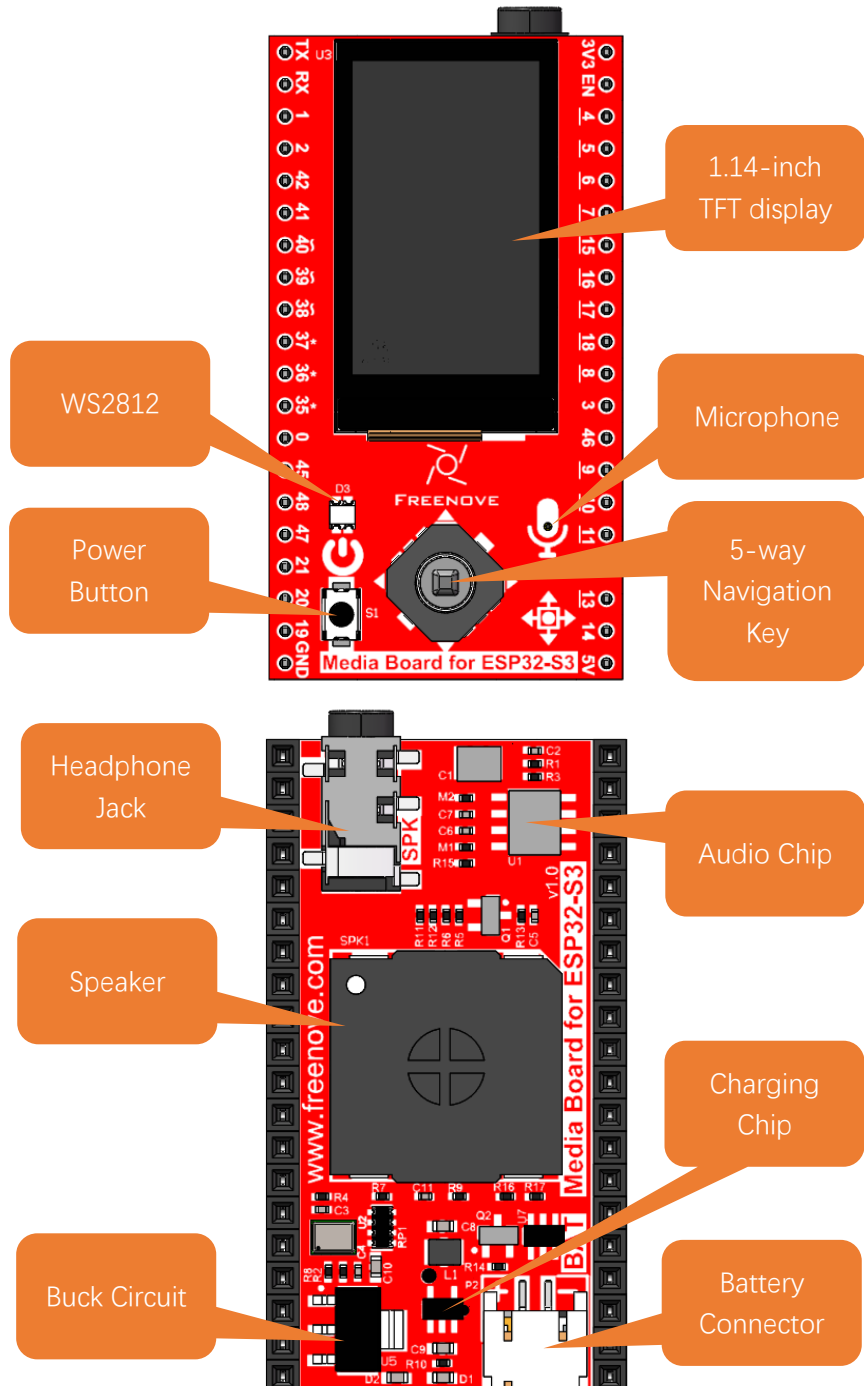
We use the Freenove ESP32-S3 Board as the main control board, which integrates 16Mb Flash and 8Mb PSRAM.



For more information about Freenove ESP32-S3 Board, please refer to https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_ESP32_S3

Audio Circuit

The audio circuit board integrates multiple functional hardware modules, including a 1.14-inch TFT display, a microphone, a battery charging circuit, an audio processing circuit, a headphone jack, and an integrated speaker, as illustrated in the figures below.



Install CH343 Driver (Required)

ESP32-S3 WROOM uses CH343 to download codes. So before using it, we need to install CH343 driver in our computers.

Windows

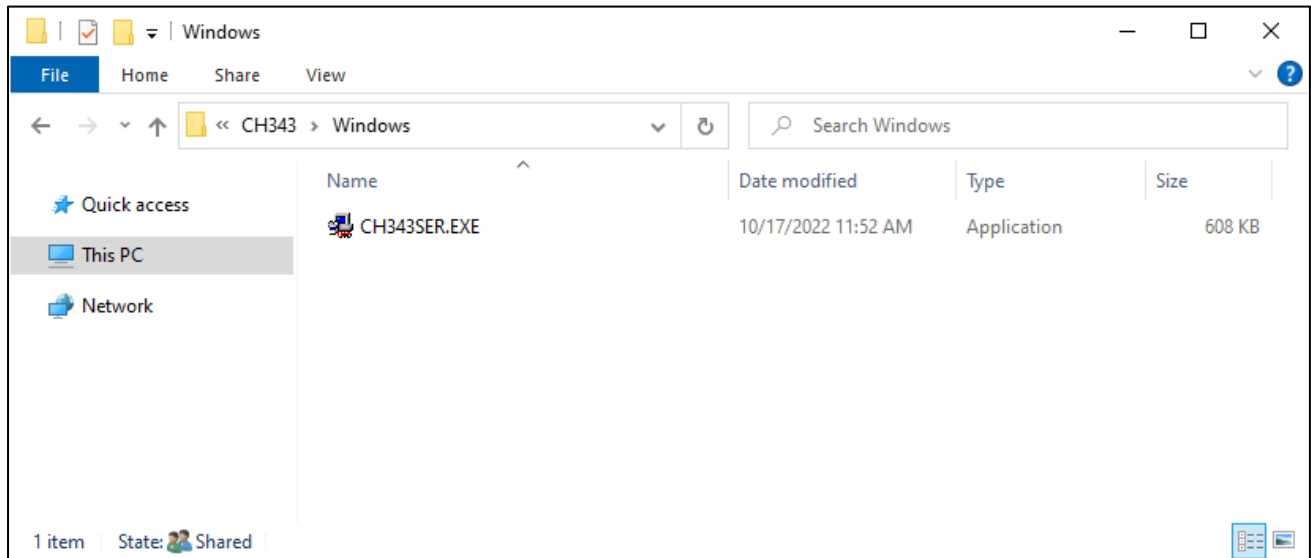
- First, download CH343 driver, click <https://www.wch-ic.com/products/CH343.html?> to download the appropriate one based on your operating system.

keyword ch343				
Downloads(8)				
file category	file content	version	upload time	
DataSheet				
CH343DS1.PDF	CH343 datasheet, USB to single serial port, supports up to 6M baud rate, serial port signals support 5V/3.3V/2.5V/1.8V, built-in crystal oscillator. CH343 supports built-in CDC driver in operating system or multi-functional high-speed VCP manufacture driver.	1.5	2021-11-18	
Driver&Tools				
CH343SER.ZIP	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13	
CH343CDC.ZIP	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13	
CH343SER.EXE	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13	
CH34XSER_MAC.ZI...	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to serial port VCP vendor driver of macOS	1.7	2022-05-13	
CH343CDC.EXE	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13	
Application				
CH34xSerCfg.ZIP	USB configuration tool of Windows for CH340/CH342/CH343/CH344/CH347/CH348/CH9101/CH9102/CH9103. Via this tool, the chip's Vendor ID, product ID, maximum current value, BCD version	1.2	2022-05-24	

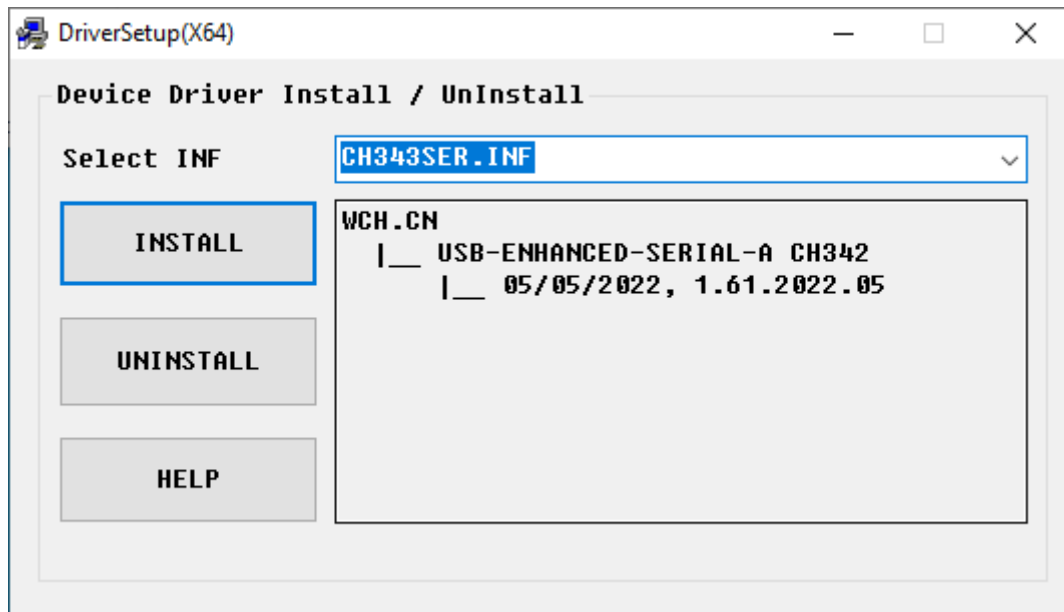
If you would not like to download the installation package, you can open “**Freenove_Media_Kit_for_ESP32-S3/CH343**”, we have prepared the installation package.

 Linux	10/17/2022 1:30 PM	File folder
 MAC	10/17/2022 1:30 PM	File folder
 Windows	10/17/2022 1:30 PM	File folder

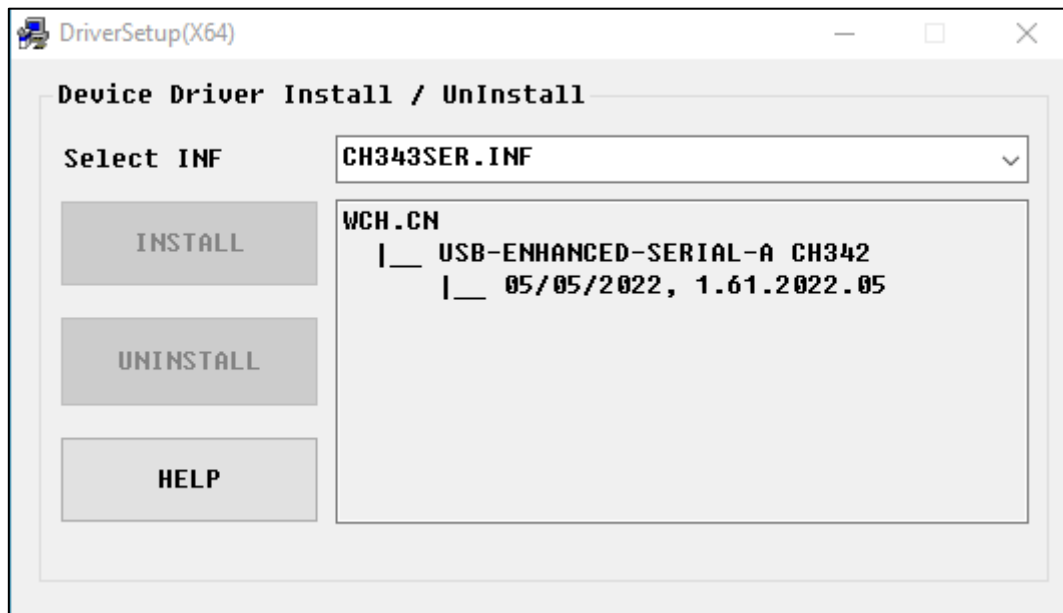
2. Open the folder “**Freenove_Media_Kit_for_ESP32-S3/CH343/Windows/**”



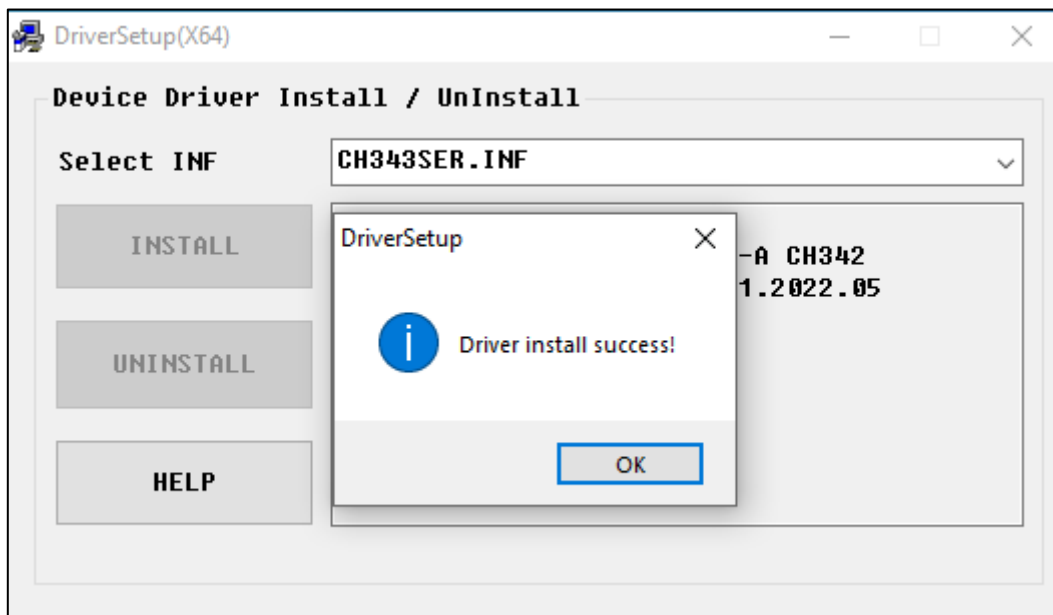
3. Double click “**CH343SER.EXE**”.



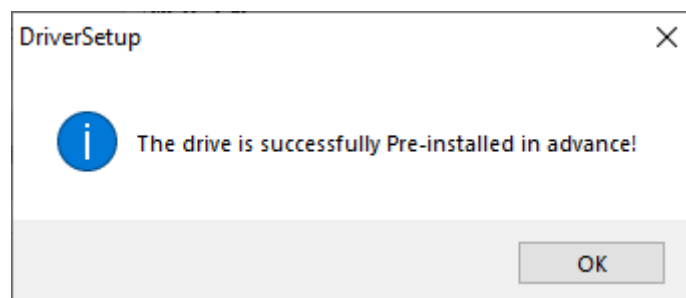
4. Click "INSTALL" and wait for the installation to complete.



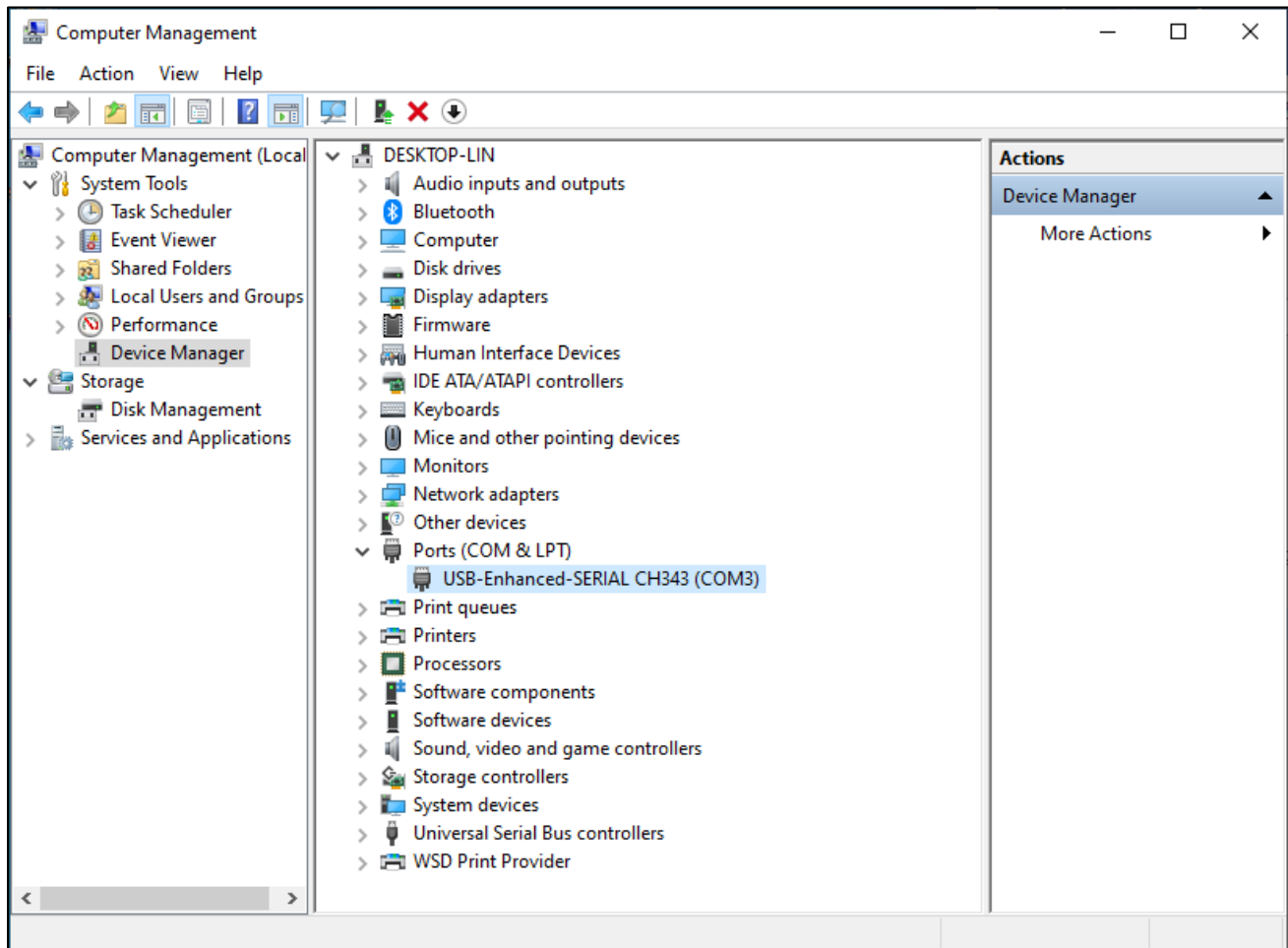
5. If the driver is successfully installed, you should see the following prompt.



Note: If you see "The drive is successfully Pre-installed in advance", it indicates the installation fails. Please make sure you use the USB data cable, not a charging cable.



6. When ESP32-S3 WROOM is connected to computer, select "This PC", right-click to select "Manage" and click "Device Manager" in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH343 has been installed successfully. Close all dialog boxes.

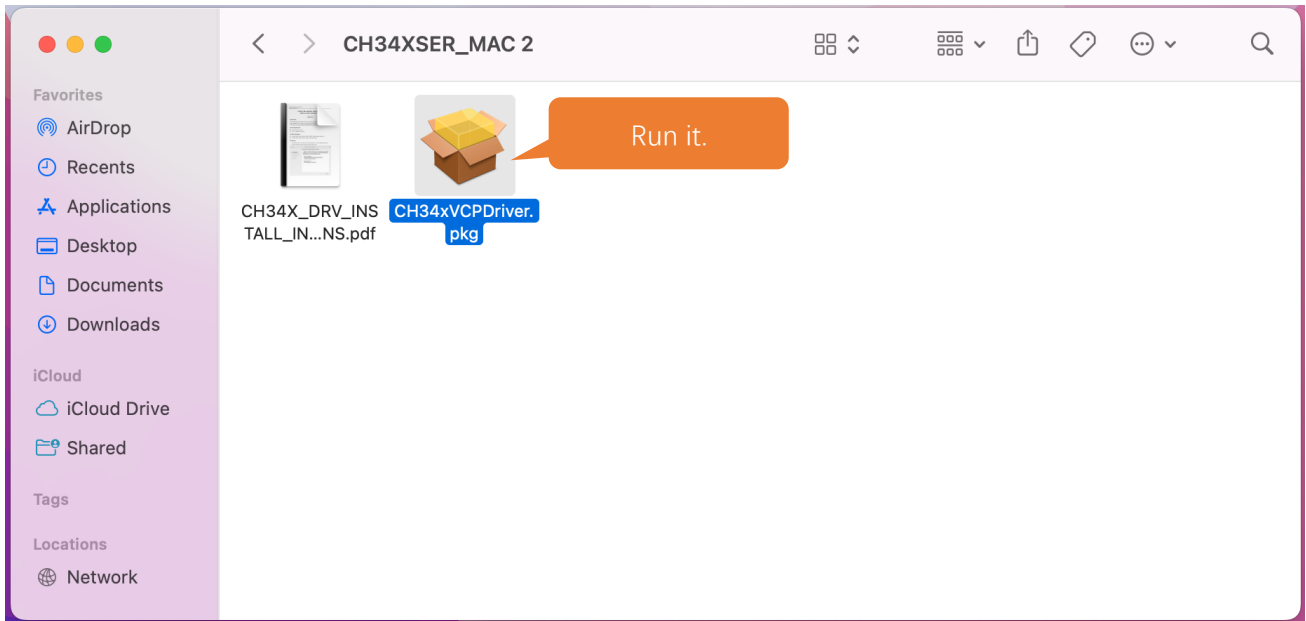
MAC

First, download CH343 driver, click <http://www.wch-ic.com/search?t=all&q=ch343> to download the appropriate one based on your operating system.

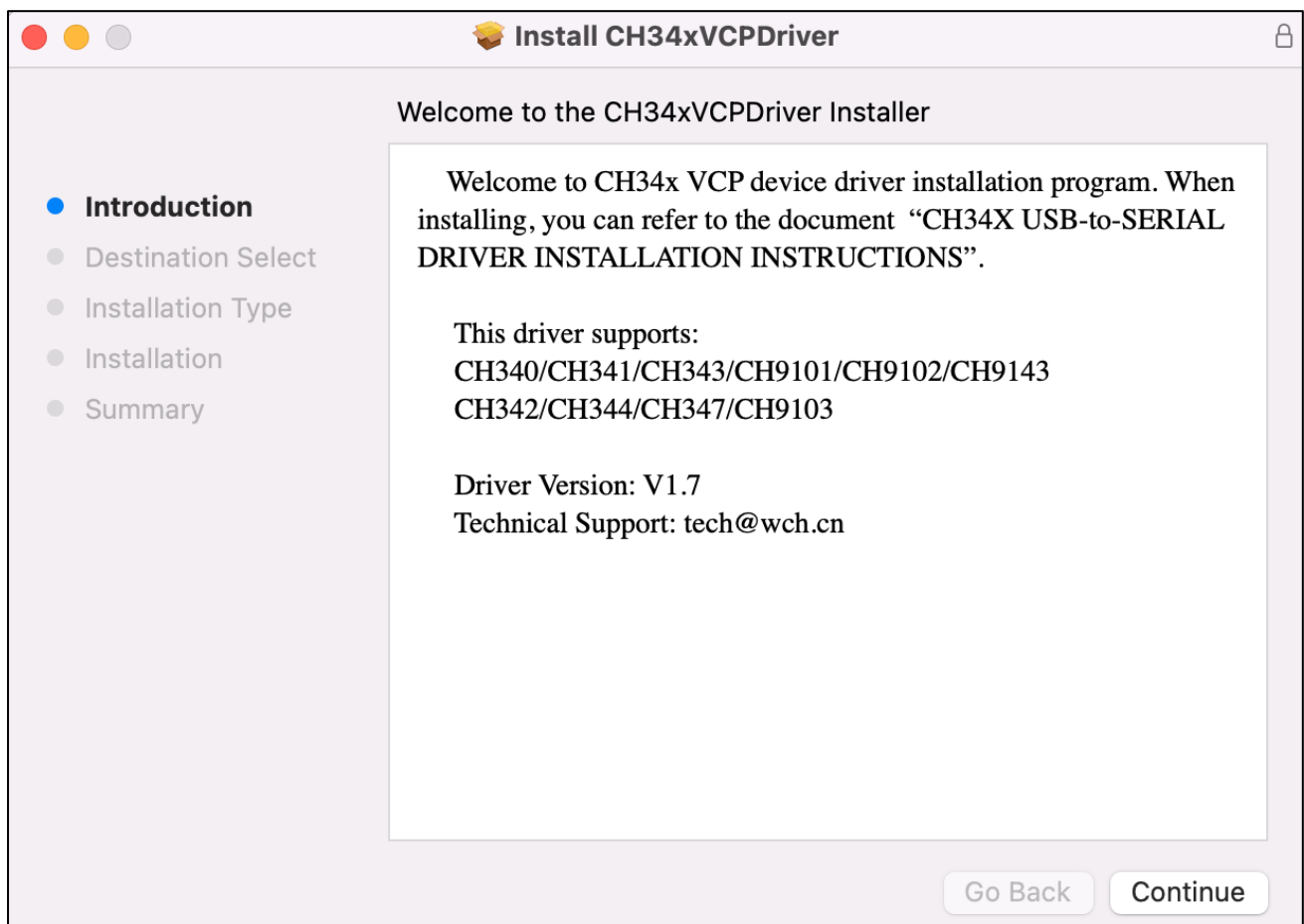
keyword ch343				
Downloads(8)				
file category	file content	version	upload time	
DataSheet				
CH343DS1.PDF	CH343 datasheet, USB to single serial port, supports up to 6M baud rate, serial port signals support 5V/3.3V/2.5V/1.8V, built-in crystal oscillator. CH343 supports built-in CDC driver in operating system or multi-functional high-speed VCP manufacture driver.	1.5	2021-11-18	
Driver&Tools				
CH343SER.ZIP	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13	
CH343CDC.ZIP	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13	
CH343SER.EXE	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13	
CH34XSER_MAC.ZI...	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to serial port VCP vendor driver of macOS	1.7	2022-05-13	
CH343CDC.EXE	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13	
Application				
CH34xSerCfg.ZIP	USB configuration tool of Windows for CH340/CH342/CH343/CH344/CH347/CH348/CH9101/CH9102/CH9103. Via this tool, the chip's Vendor ID, product ID, maximum current value, BCD version	1.2	2022-05-24	

If you would not like to download the installation package, you can open “**Freenove_Media_Kit_for_ESP32-S3/CH343**”, we have prepared the installation package.

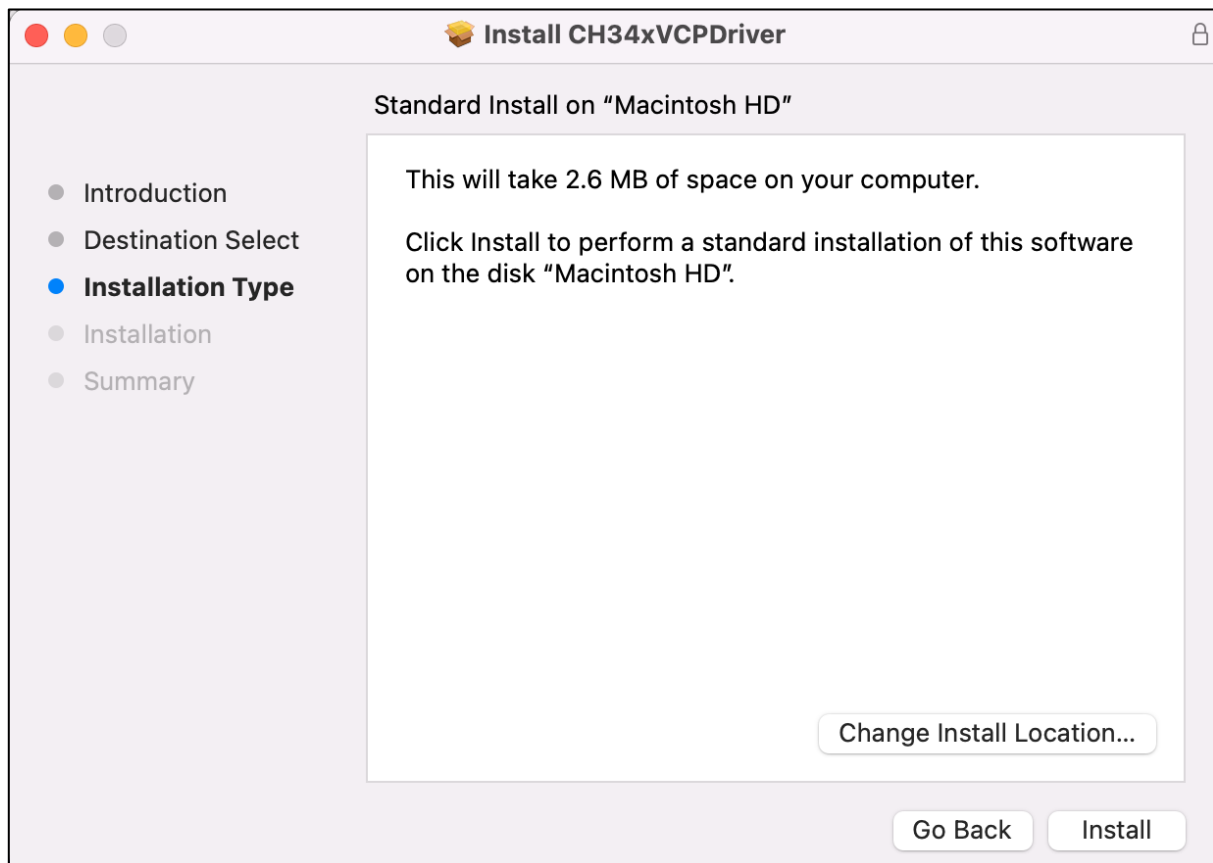
Second, open the folder “**Freenove_Media_Kit_for_ESP32-S3/CH343/MAC/**”



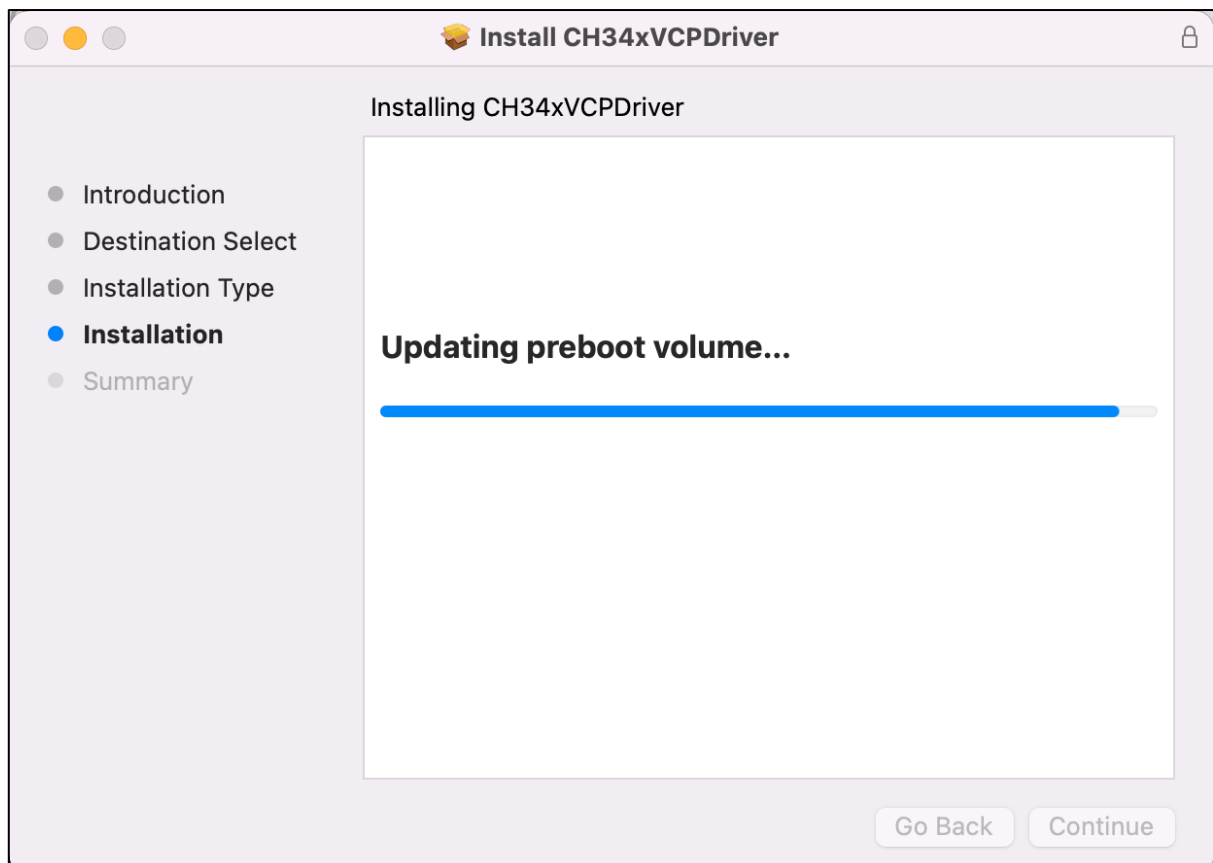
Third, click Continue.



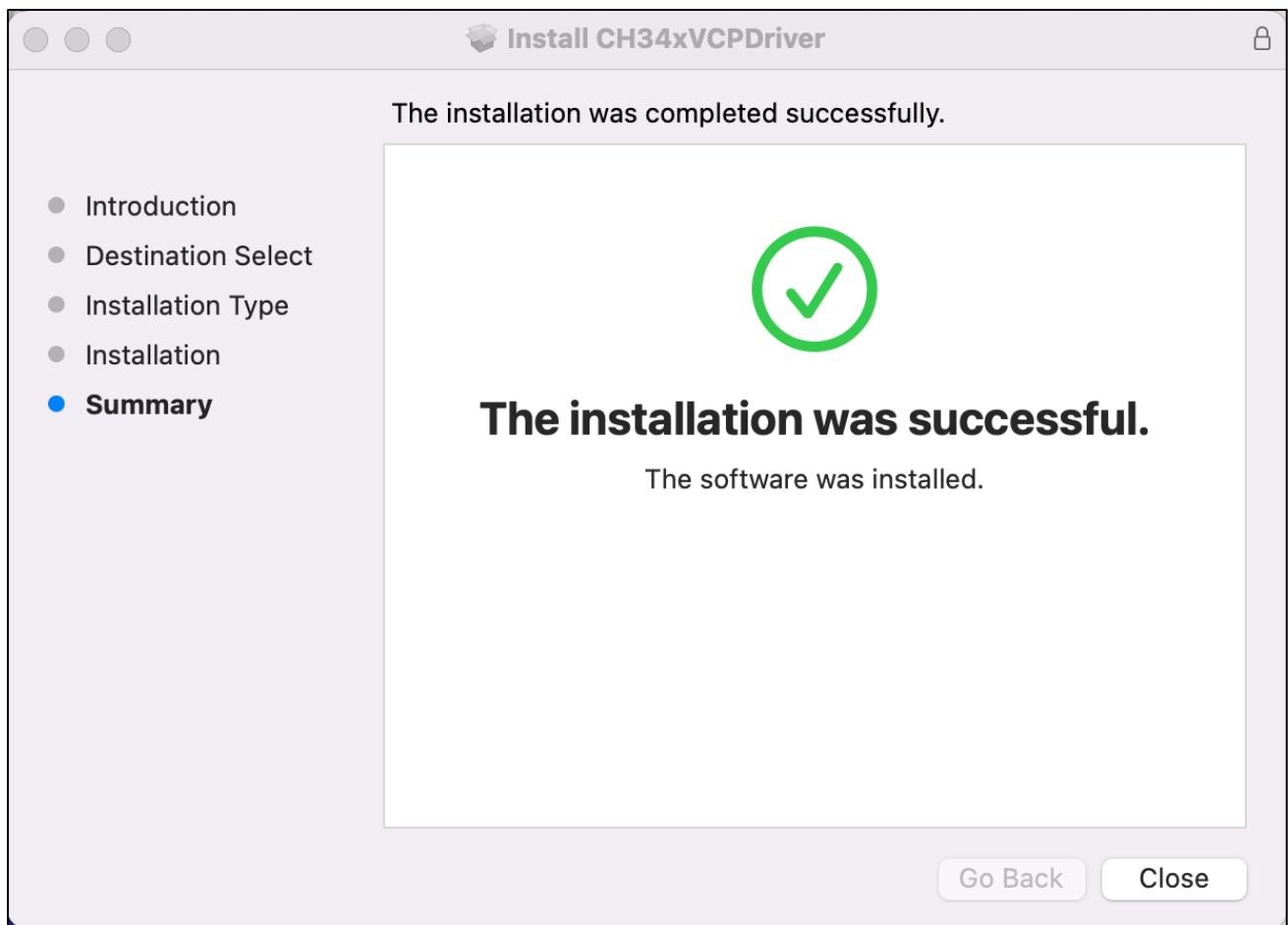
Fourth, click Install.



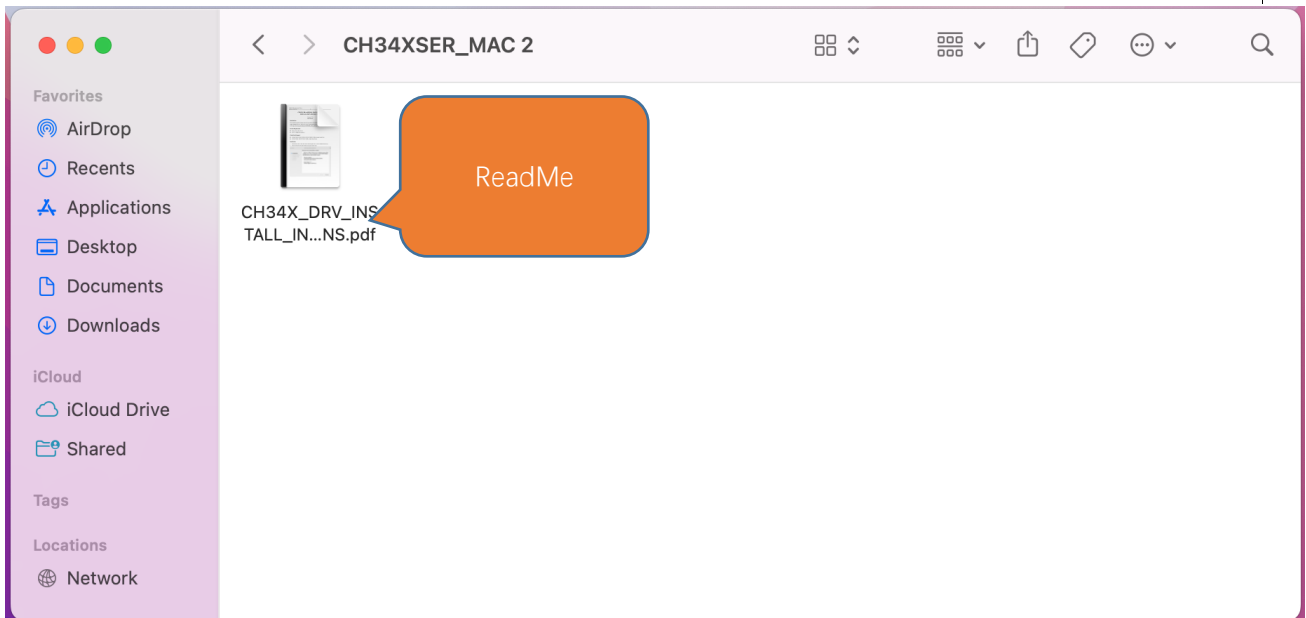
Then, waiting Finish.



Finally, restart your PC.

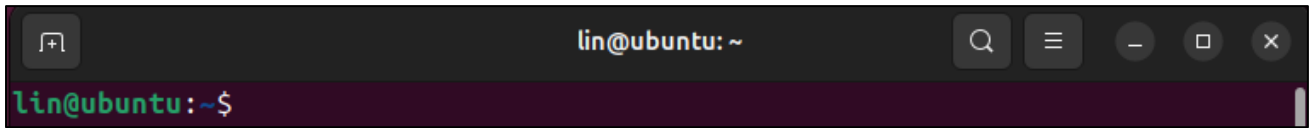


If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.



Linux

Here we take Ubuntu as an example. Open the terminal in Linux system.



Check the port with the command "lsusb".

```
lsusb
```

```
ls /dev/tty*
```

```
lin@lin-virtual-machine:~$ lsusb
```

```
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

```
Bus 001 Device 004: ID 1a86:55d3 QinHeng Electronics USB Single Serial
```

```
Bus 001 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
```

```
Bus 001 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
```

```
Bus 001 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

```
lin@lin-virtual-machine:~$ ls /dev/tty*
```

/dev/tty	/dev/tty23	/dev/tty39	/dev/tty54	/dev/ttyS1	/dev/ttyS25
/dev/tty0	/dev/tty24	/dev/tty4	/dev/tty55	/dev/ttyS10	/dev/ttyS26
/dev/tty1	/dev/tty25	/dev/tty40	/dev/tty56	/dev/ttyS11	/dev/ttyS27
/dev/tty10	/dev/tty26	/dev/tty41	/dev/tty57	/dev/ttyS12	/dev/ttyS28
/dev/tty11	/dev/tty27	/dev/tty42	/dev/tty58	/dev/ttyS13	/dev/ttyS29
/dev/tty12	/dev/tty28	/dev/tty43	/dev/tty59	/dev/ttyS14	/dev/ttyS3
/dev/tty13	/dev/tty29	/dev/tty44	/dev/tty6	/dev/ttyS15	/dev/ttyS30
/dev/tty14	/dev/tty3	/dev/tty45	/dev/tty60	/dev/ttyS16	/dev/ttyS31
/dev/tty15	/dev/tty30	/dev/tty46	/dev/tty61	/dev/ttyS17	/dev/ttyS4
/dev/tty16	/dev/tty31	/dev/tty47	/dev/tty62	/dev/ttyS18	/dev/ttyS5
/dev/tty17	/dev/tty32	/dev/tty48	/dev/tty63	/dev/ttyS19	/dev/ttyS6
/dev/tty18	/dev/tty33	/dev/tty49	/dev/tty7	/dev/ttyS2	/dev/ttyS7
/dev/tty19	/dev/tty34	/dev/tty5	/dev/tty8	/dev/ttyS20	/dev/ttyS8
/dev/tty2	/dev/tty35	/dev/tty50	/dev/tty9	/dev/ttyS21	/dev/ttyS9
/dev/tty20	/dev/tty36	/dev/tty51	/dev/ttyACM0	/dev/ttyS22	
/dev/tty21	/dev/tty37	/dev/tty52	/dev/ttyprintk	/dev/ttyS23	
/dev/tty22	/dev/tty38	/dev/tty53	/dev/ttyS0	/dev/ttyS24	

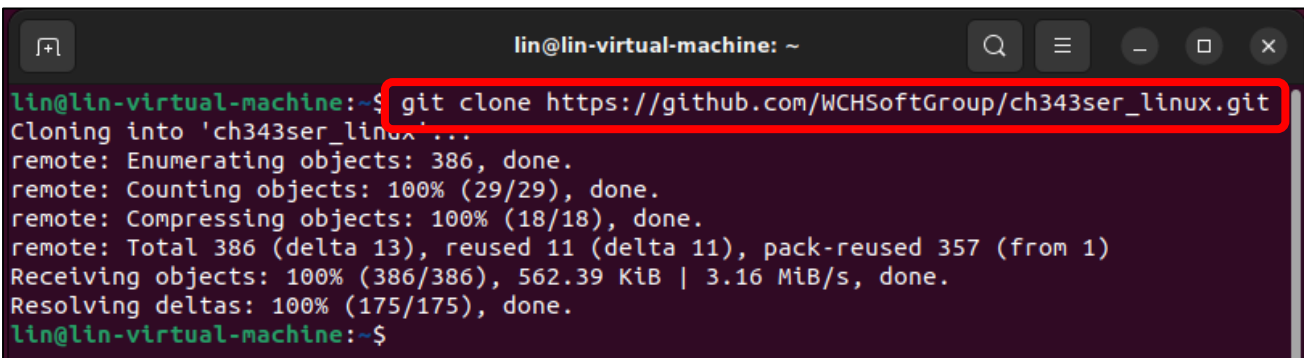
```
lin@lin-virtual-machine:~$
```

Generally, the CH34x driver is included in modern Linux kernels, so it should work automatically when the device is connected

If your computer does not have the CH343 driver, you can follow the steps below to install it. If your computer recognizes the CH343 driver, you may skip the following steps.

Run the following command to download the driver.

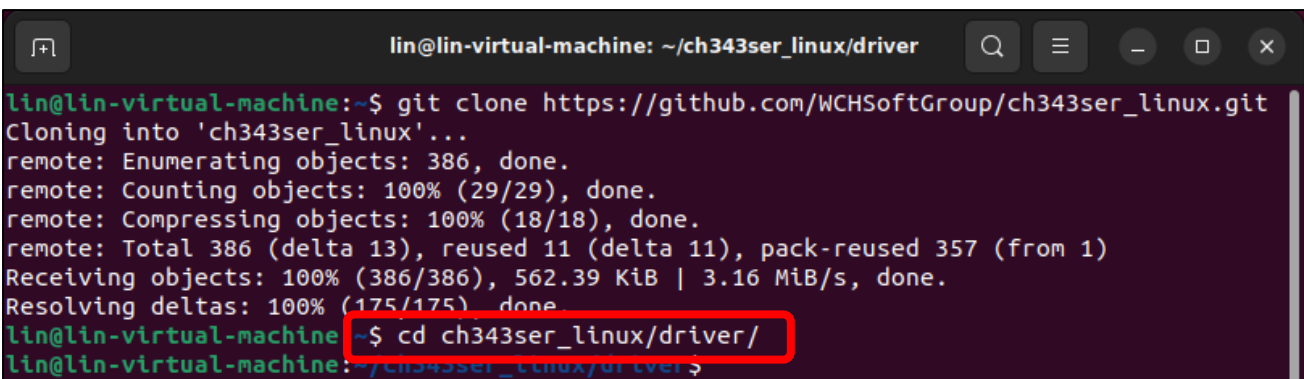
```
git clone https://github.com/WCHSoftGroup/ch343ser_linux.git
```



```
lin@lin-virtual-machine: ~  
lin@lin-virtual-machine:~$ git clone https://github.com/WCHSoftGroup/ch343ser_linux.git  
Cloning into 'ch343ser_linux'...  
remote: Enumerating objects: 386, done.  
remote: Counting objects: 100% (29/29), done.  
remote: Compressing objects: 100% (18/18), done.  
remote: Total 386 (delta 13), reused 11 (delta 11), pack-reused 357 (from 1)  
Receiving objects: 100% (386/386), 562.39 KiB | 3.16 MiB/s, done.  
Resolving deltas: 100% (175/175), done.  
lin@lin-virtual-machine:~$
```

Enter the folder where the driver locates.

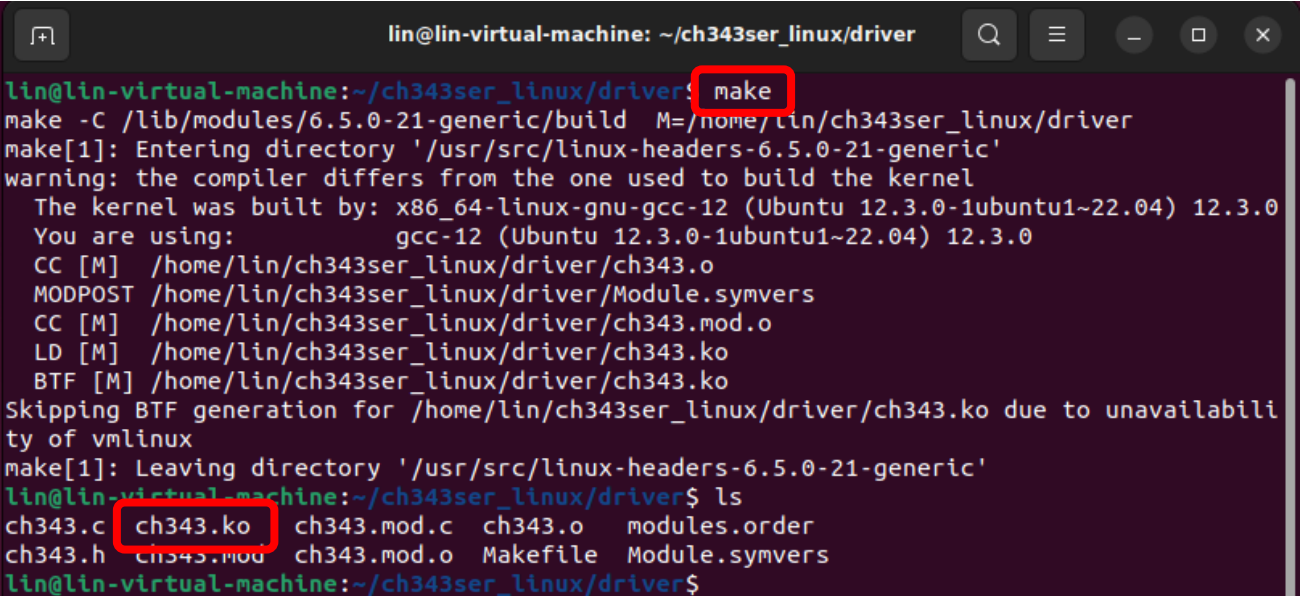
```
cd ch343ser_linux/driver/
```



```
lin@lin-virtual-machine: ~/ch343ser_linux/driver  
lin@lin-virtual-machine:~$ git clone https://github.com/WCHSoftGroup/ch343ser_linux.git  
Cloning into 'ch343ser_linux'...  
remote: Enumerating objects: 386, done.  
remote: Counting objects: 100% (29/29), done.  
remote: Compressing objects: 100% (18/18), done.  
remote: Total 386 (delta 13), reused 11 (delta 11), pack-reused 357 (from 1)  
Receiving objects: 100% (386/386), 562.39 KiB | 3.16 MiB/s, done.  
Resolving deltas: 100% (175/175), done.  
lin@lin-virtual-machine:~/ch343ser_linux/driver$ cd ch343ser_linux/driver/  
lin@lin-virtual-machine:~/ch343ser_linux/driver$
```

Compile to generate a ch343.ko file.

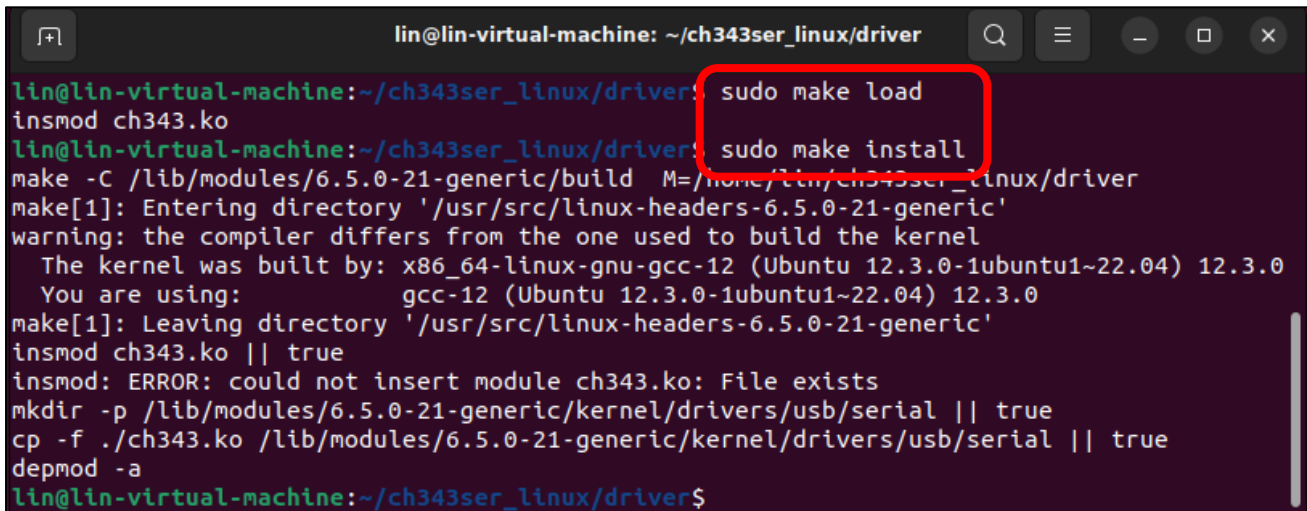
```
make
```



```
lin@lin-virtual-machine: ~/ch343ser_linux/driver  
lin@lin-virtual-machine:~/ch343ser_linux/driver$ make  
make -C /lib/modules/6.5.0-21-generic/build M=/home/lin/ch343ser_linux/driver  
make[1]: Entering directory '/usr/src/linux-headers-6.5.0-21-generic'  
warning: the compiler differs from the one used to build the kernel  
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0  
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0  
CC [M] /home/lin/ch343ser_linux/driver/ch343.o  
MODPOST /home/lin/ch343ser_linux/driver/Module.symvers  
CC [M] /home/lin/ch343ser_linux/driver/ch343.mod.o  
LD [M] /home/lin/ch343ser_linux/driver/ch343.ko  
BTF [M] /home/lin/ch343ser_linux/driver/ch343.ko  
Skipping BTF generation for /home/lin/ch343ser_linux/driver/ch343.ko due to unavailability of vmlinux  
make[1]: Leaving directory '/usr/src/linux-headers-6.5.0-21-generic'  
lin@lin-virtual-machine:~/ch343ser_linux/driver$ ls  
ch343.c ch343.ko ch343.mod.c ch343.o modules.order  
ch343.h ch343.mod ch343.mod.o Makefile Module.symvers  
lin@lin-virtual-machine:~/ch343ser_linux/driver$
```


Install the ch343 chip driver.

```
sudo make load
sudo make install
```



```
lin@lin-virtual-machine: ~/ch343ser_linux/driver
lin@lin-virtual-machine:~/ch343ser_linux/driver$ sudo make load
insmod ch343.ko
lin@lin-virtual-machine:~/ch343ser_linux/driver$ sudo make install
make -C /lib/modules/6.5.0-21-generic/build M=/home/lin/ch343ser_linux/driver
make[1]: Entering directory '/usr/src/linux-headers-6.5.0-21-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
make[1]: Leaving directory '/usr/src/linux-headers-6.5.0-21-generic'
insmod ch343.ko || true
insmod: ERROR: could not insert module ch343.ko: File exists
mkdir -p /lib/modules/6.5.0-21-generic/kernel/drivers/usb/serial || true
cp -f ./ch343.ko /lib/modules/6.5.0-21-generic/kernel/drivers/usb/serial || true
depmod -a
lin@lin-virtual-machine:~/ch343ser_linux/driver$
```

Connect the ESP32S3 to your computer, run the following command and you should see the port.

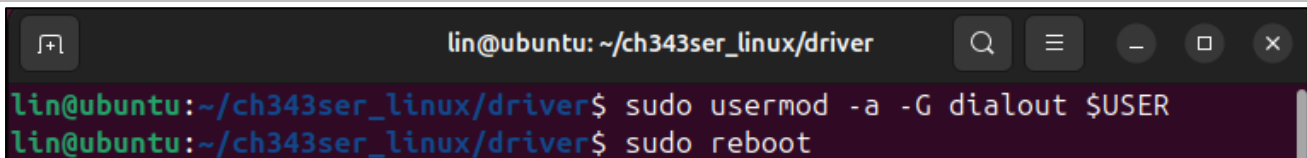
```
ls /dev/tty*
```



```
(myvenv) lin@ubuntu:~/Upload_Xiaozhi_Bin$ ls /dev/tty*
/dev/tty    /dev/tty2    /dev/tty31   /dev/tty43   /dev/tty55   /dev/ttyACM0  /dev/ttyS19  /dev/ttyS30
/dev/tty0    /dev/tty20   /dev/tty32   /dev/tty44   /dev/tty56   /dev/ttyS2    /dev/ttyS21  /dev/ttyS31
/dev/tty1    /dev/tty21   /dev/tty33   /dev/tty45   /dev/tty57   /dev/ttyS0    /dev/ttyS22  /dev/ttyS4
/dev/tty10   /dev/tty22   /dev/tty34   /dev/tty46   /dev/tty58   /dev/ttyS1    /dev/ttyS23  /dev/ttyS5
/dev/tty11   /dev/tty23   /dev/tty35   /dev/tty47   /dev/tty59   /dev/ttyS10   /dev/ttyS24  /dev/ttyS6
/dev/tty12   /dev/tty24   /dev/tty36   /dev/tty48   /dev/tty60   /dev/ttyS11   /dev/ttyS25  /dev/ttyS7
/dev/tty13   /dev/tty25   /dev/tty37   /dev/tty49   /dev/tty61   /dev/ttyS12   /dev/ttyS26  /dev/ttyS8
/dev/tty14   /dev/tty26   /dev/tty38   /dev/tty5    /dev/tty62   /dev/ttyS13   /dev/ttyS27  /dev/ttyS9
/dev/tty15   /dev/tty27   /dev/tty39   /dev/tty50   /dev/tty63   /dev/ttyS14   /dev/ttyS28  /dev/ttyS10
/dev/tty16   /dev/tty28   /dev/tty4    /dev/tty51   /dev/tty64   /dev/ttyS15   /dev/ttyS29  /dev/ttyS11
/dev/tty17   /dev/tty29   /dev/tty40   /dev/tty52   /dev/tty65   /dev/ttyS16   /dev/ttyS30  /dev/ttyS12
/dev/tty18   /dev/tty3    /dev/tty41   /dev/tty53   /dev/tty66   /dev/ttyS17   /dev/ttyS31  /dev/ttyS13
/dev/tty19   /dev/tty30   /dev/tty42   /dev/tty54   /dev/tty67   /dev/ttyS18   /dev/ttyS32  /dev/ttyS14
(myvenv) lin@ubuntu:~/Upload_Xiaozhi_Bin$
```

In Linux, higher permissions are required to access "ttyACM0," so privilege escalation commands must be used.

```
sudo usermod -a -G dialout $USER
sudo reboot
```



```
lin@ubuntu:~/ch343ser_linux/driver$ sudo usermod -a -G dialout $USER
lin@ubuntu:~/ch343ser_linux/driver$ sudo reboot
```

Reboot the system to have the configuration take effect.

OpenAI Code

Visual Studio Code

As we use Visual Studio Code to compile and upload code, we need to install the software before proceeding.

Windows

First, download Visual Studio Code by visiting <https://code.visualstudio.com/Download>. Choose the appropriate version for your operating system, then download and install it

Visual Studio Code

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows
Windows 10, 11

User Installer [x64](#) [Arm64](#)
System [x64](#) [Arm64](#)
Installer [x64](#) [Arm64](#)
.zip [x64](#) [Arm64](#)
CLI [x64](#) [Arm64](#)



↓ .deb
Debian, Ubuntu

↓ .rpm
Red Hat, Fedora, SUSE

.deb [x64](#) [Arm32](#) [Arm64](#)
.rpm [x64](#) [Arm32](#) [Arm64](#)
.tar.gz [x64](#) [Arm32](#) [Arm64](#)
Snap [Snap Store](#)
CLI [x64](#) [Arm32](#) [Arm64](#)



↓ Mac
macOS 10.15+

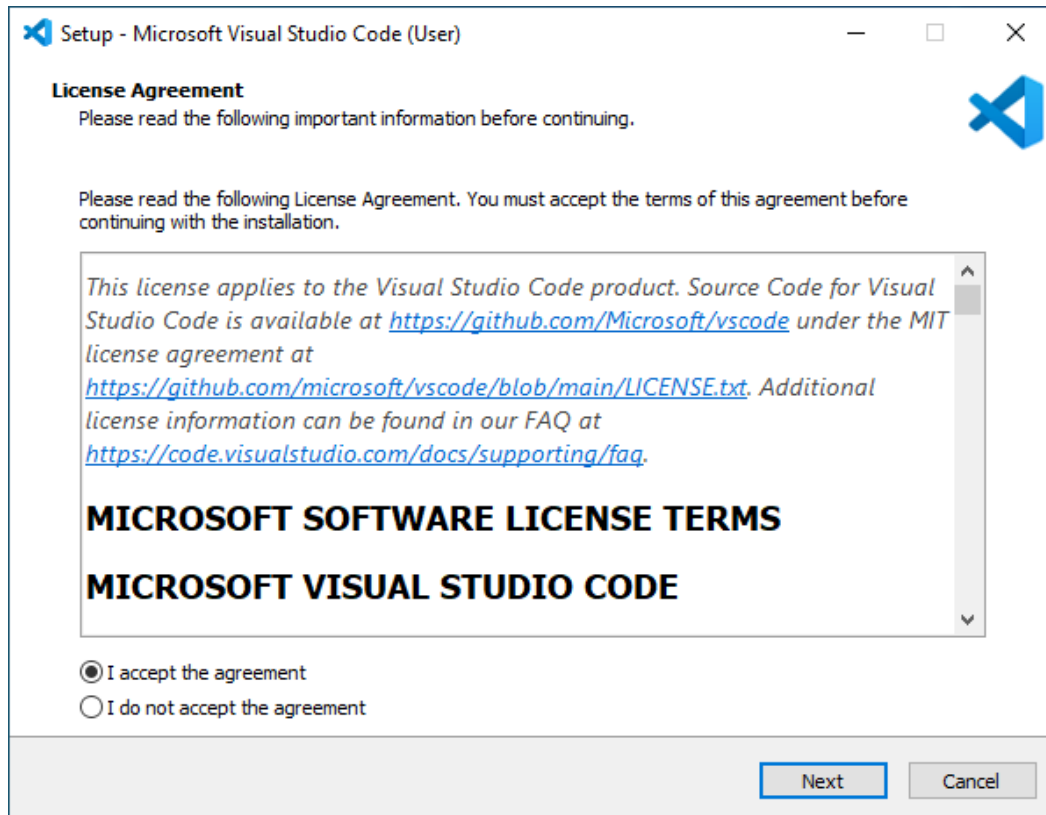
.zip [Intel chip](#) [Apple silicon](#) [Universal](#)
CLI [Intel chip](#) [Apple silicon](#)

By downloading and using Visual Studio Code, you agree to the [license terms](#) and [privacy statement](#).

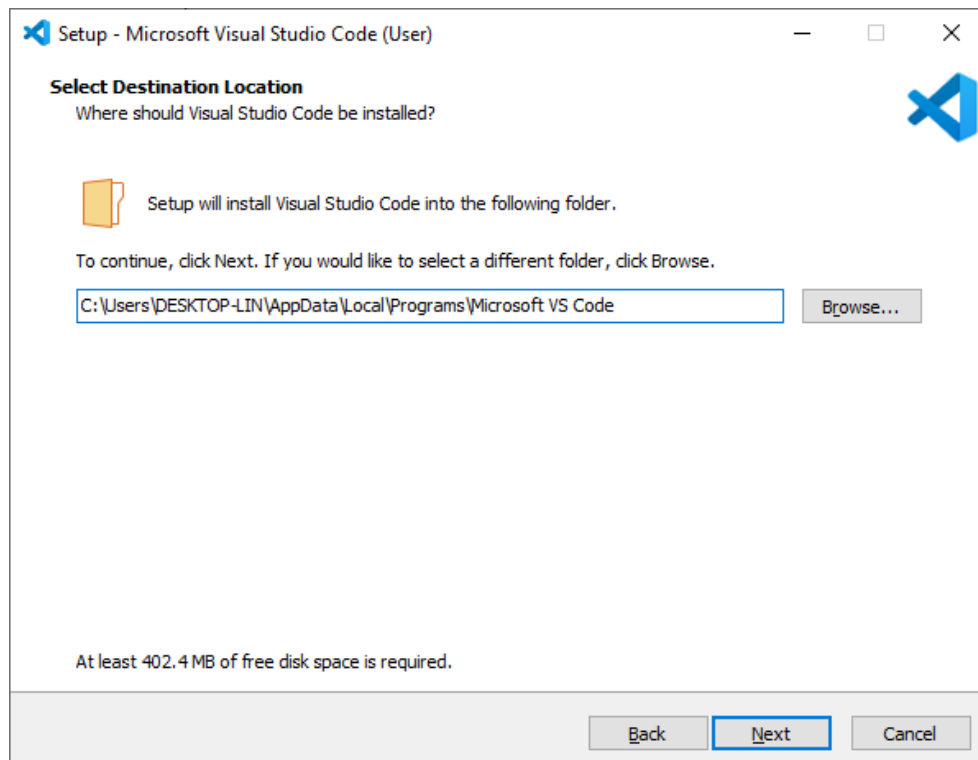
Double-click the downloaded .exe file to run it.

Check the box for "I accept the agreement."

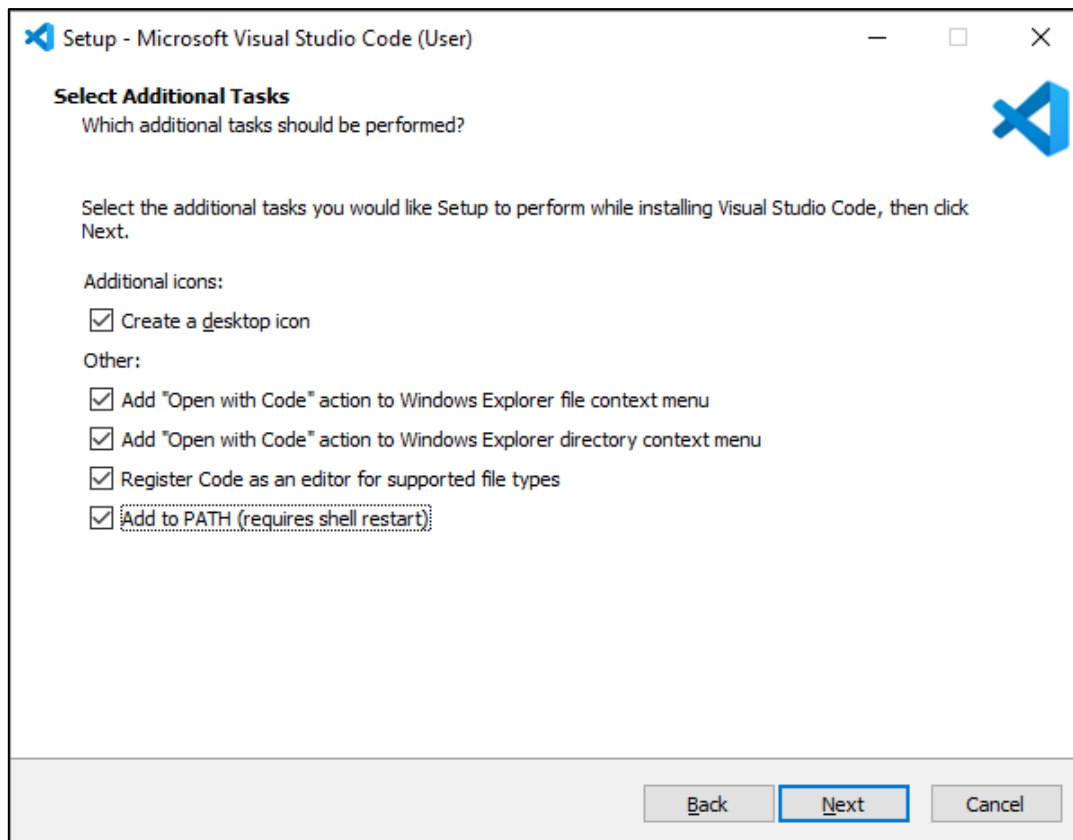
Then click Next.



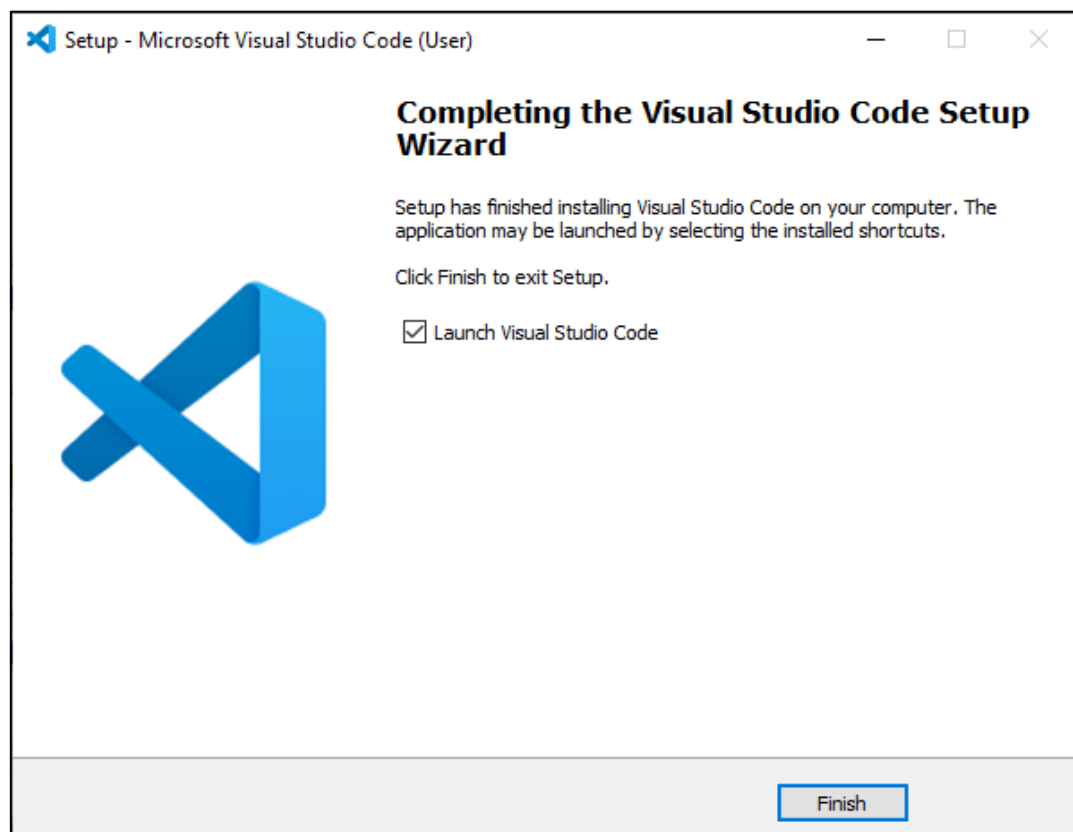
The installation location can be left as the default or changed to a desired path. After that, proceed by clicking Next repeatedly.



On this screen, verify that "Add to PATH" is selected. If unchecked, enable it. Proceed by clicking Next repeatedly to finish the installation.



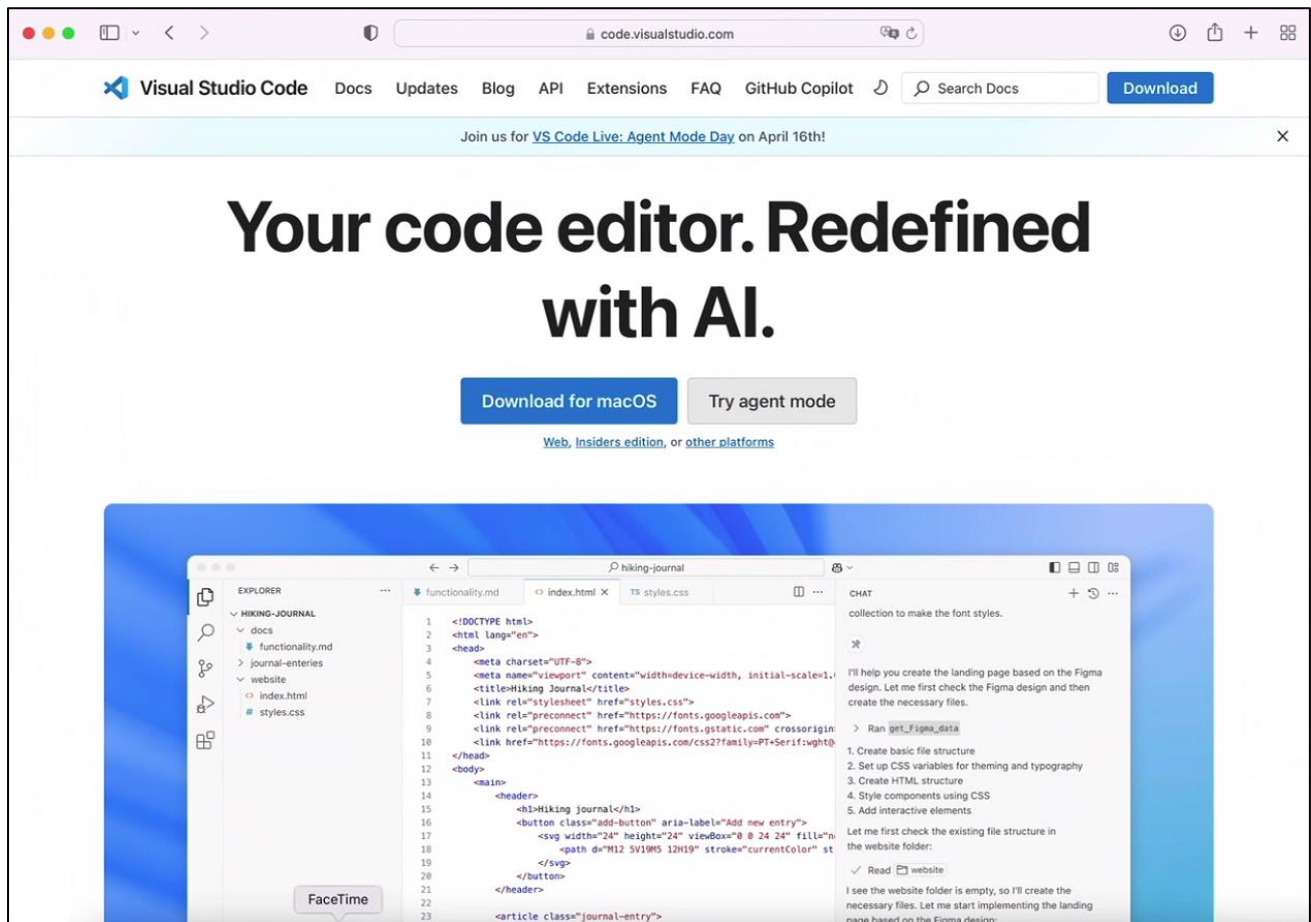
The installation is now complete, as shown in the image below.



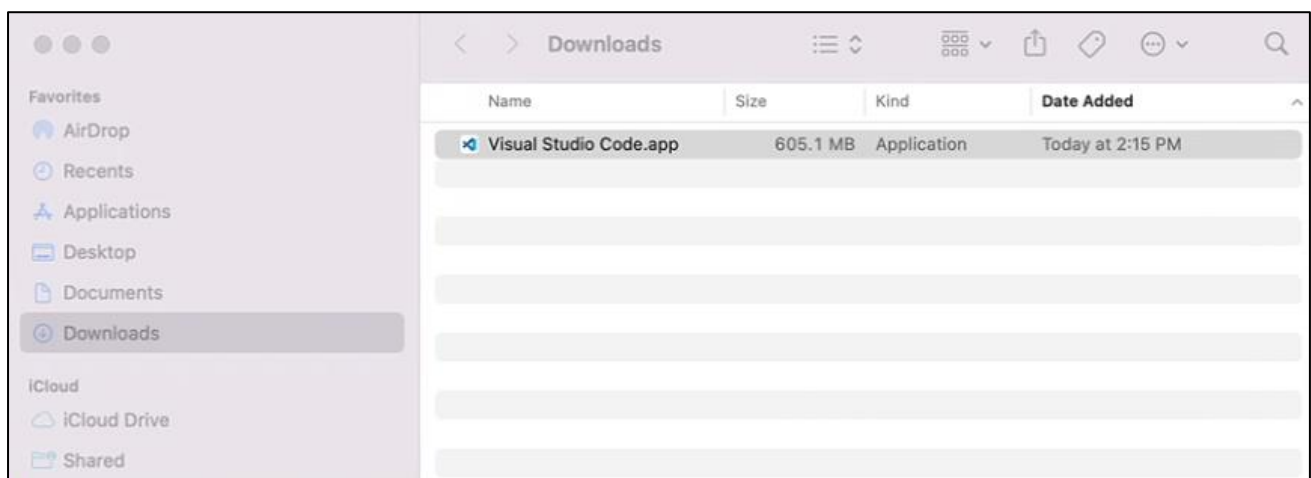
Mac

Typically, MacOS comes with Visual Studio Code pre-installed. If your computer does not have it, please install it first.

Visit <https://code.visualstudio.com> and click "Download for macOS".



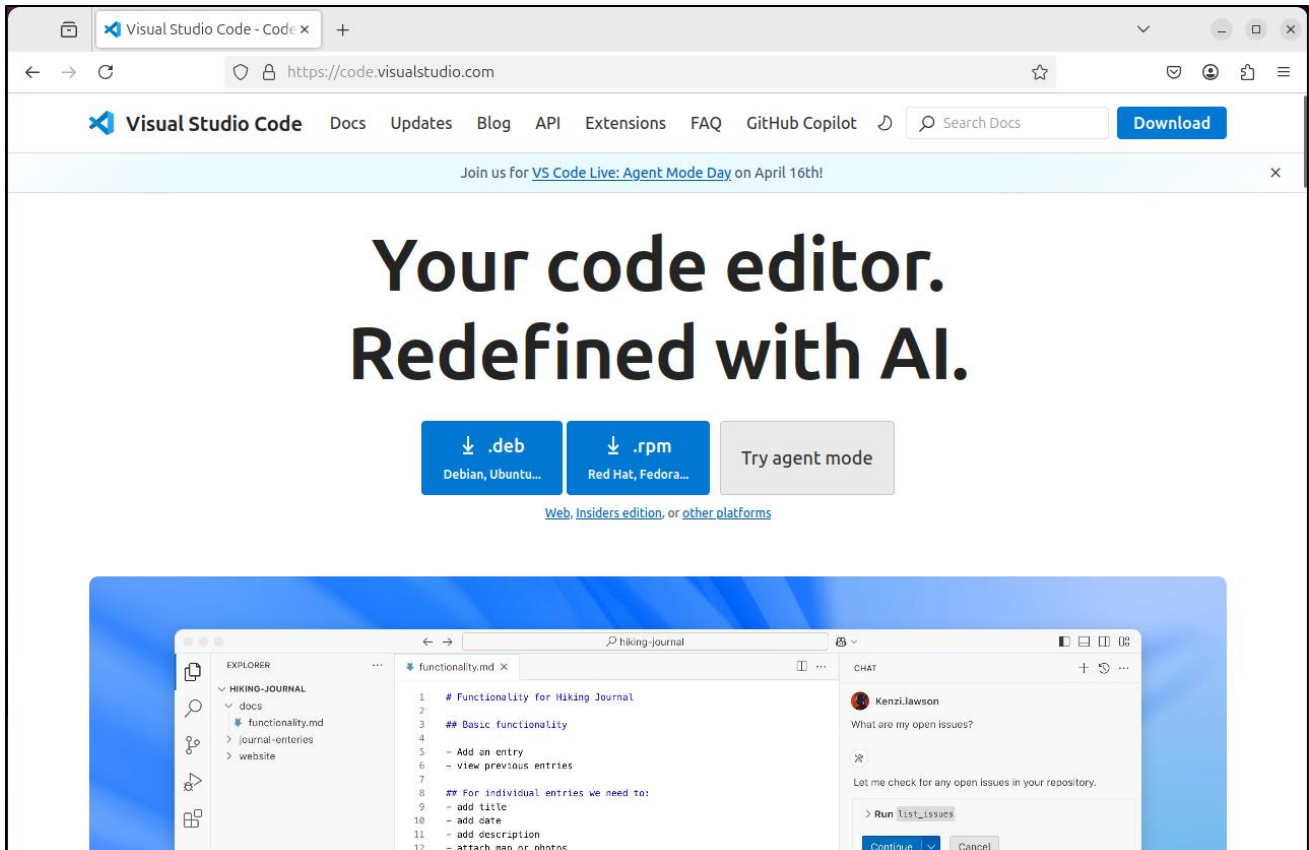
Double click to run the program.



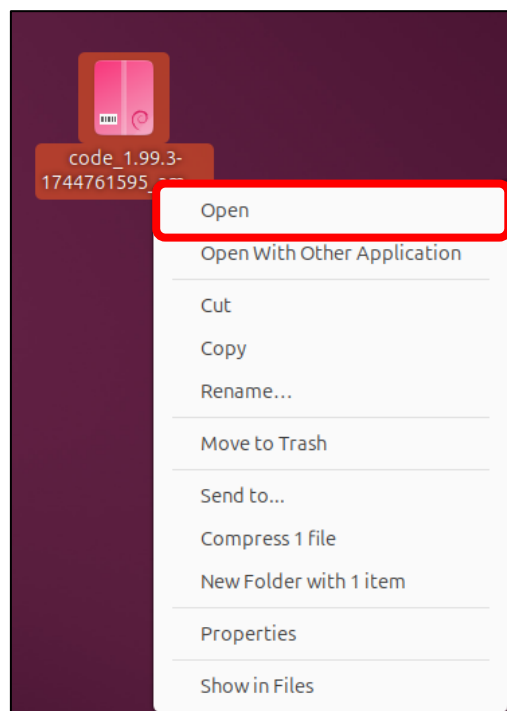
Linux

If your computer does not have Visual Studio Code, please install it first.

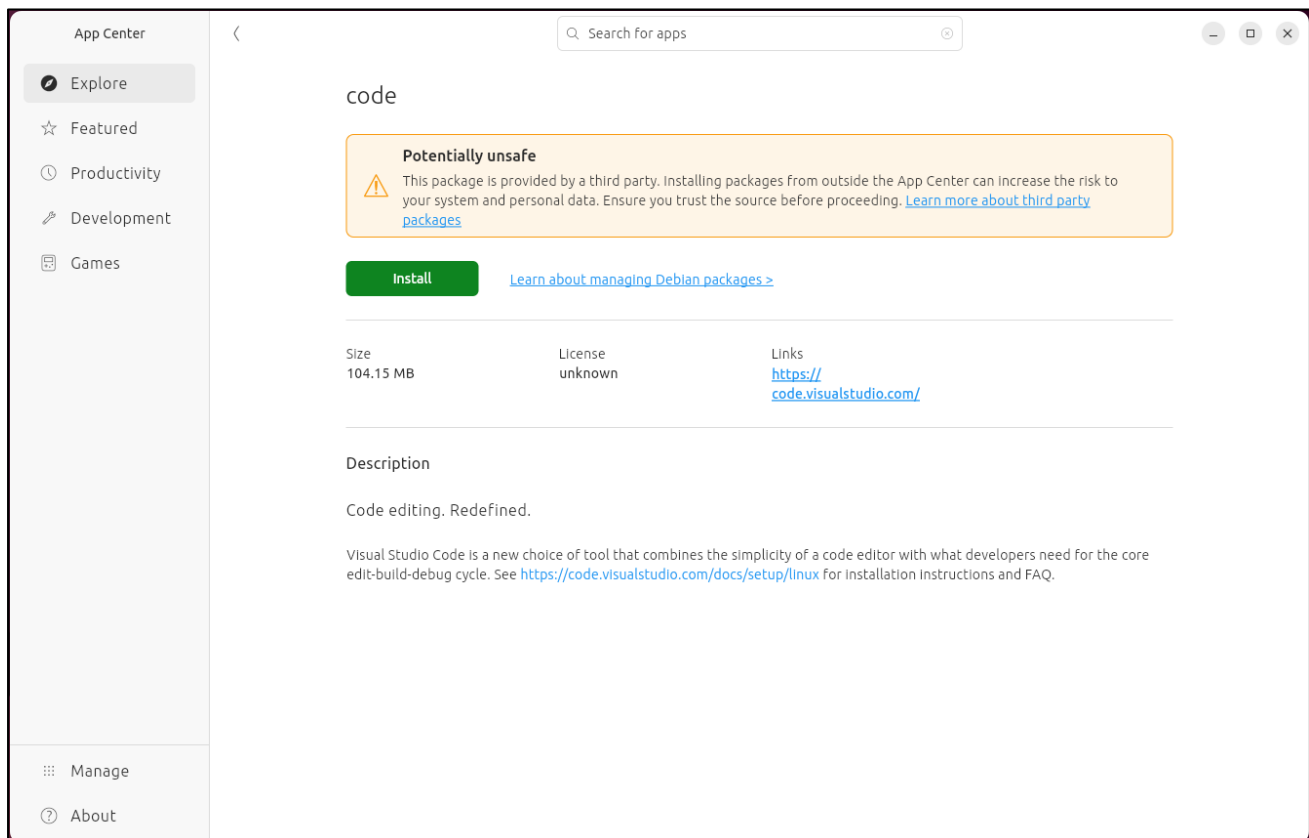
Visit <https://code.visualstudio.com> and click ".deb" .



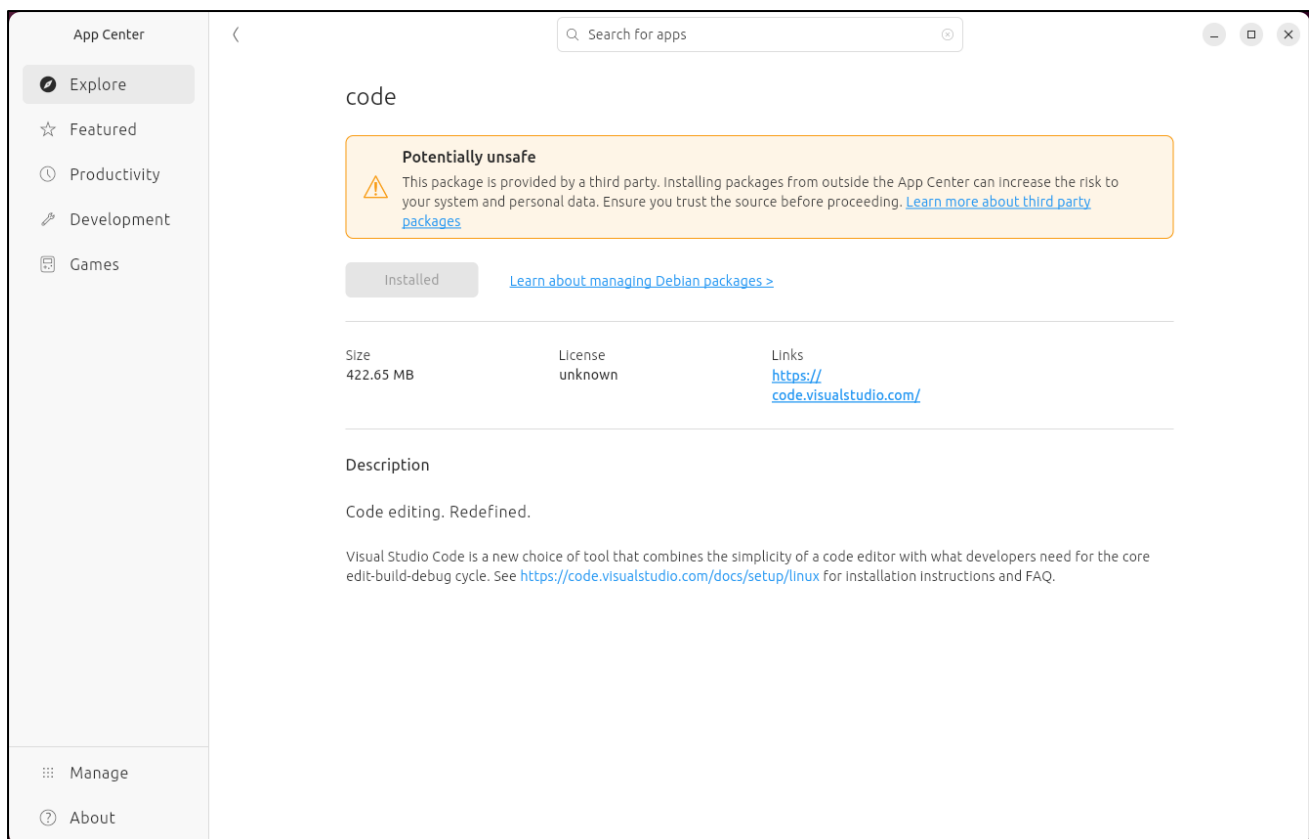
Open the downloaded "code_xxx.deb" file.



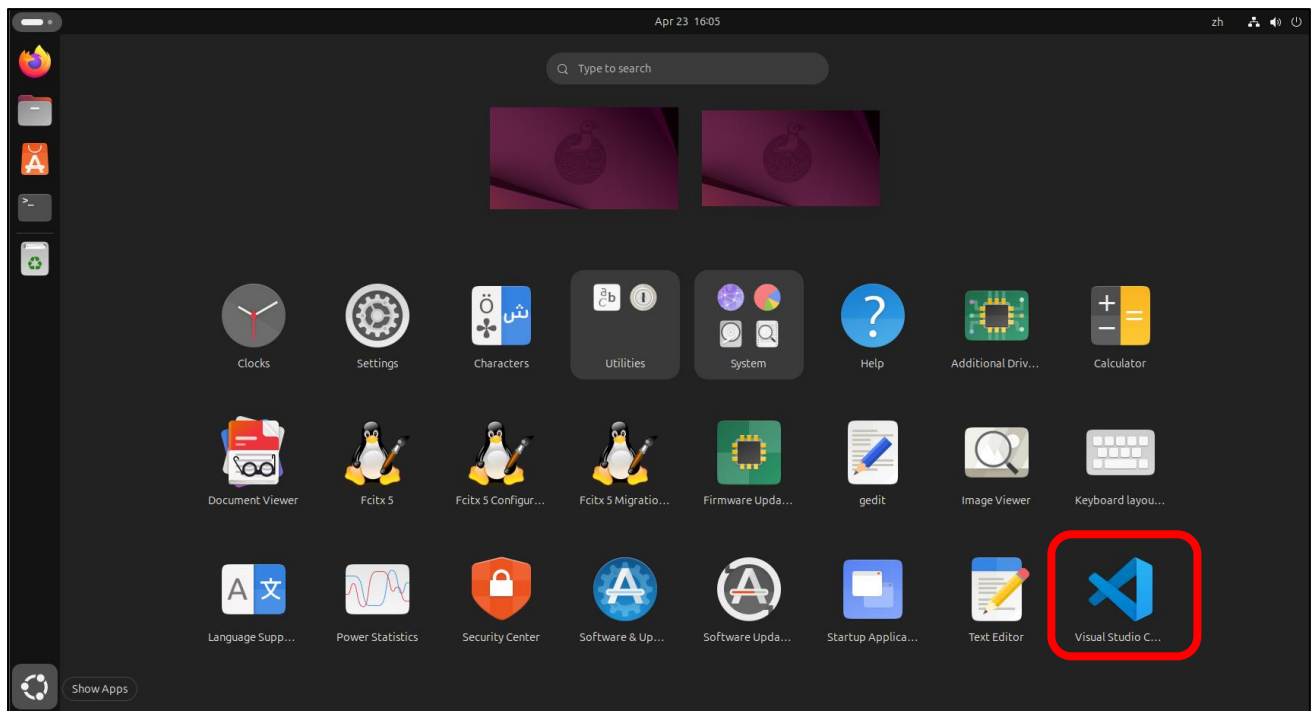
Click “Install” to install Visual Studio Code.



Wait for the installation to complete. Once finished, it should look like the image below.



Click Show Apps and you can see the Visual Studio Code is in the system.

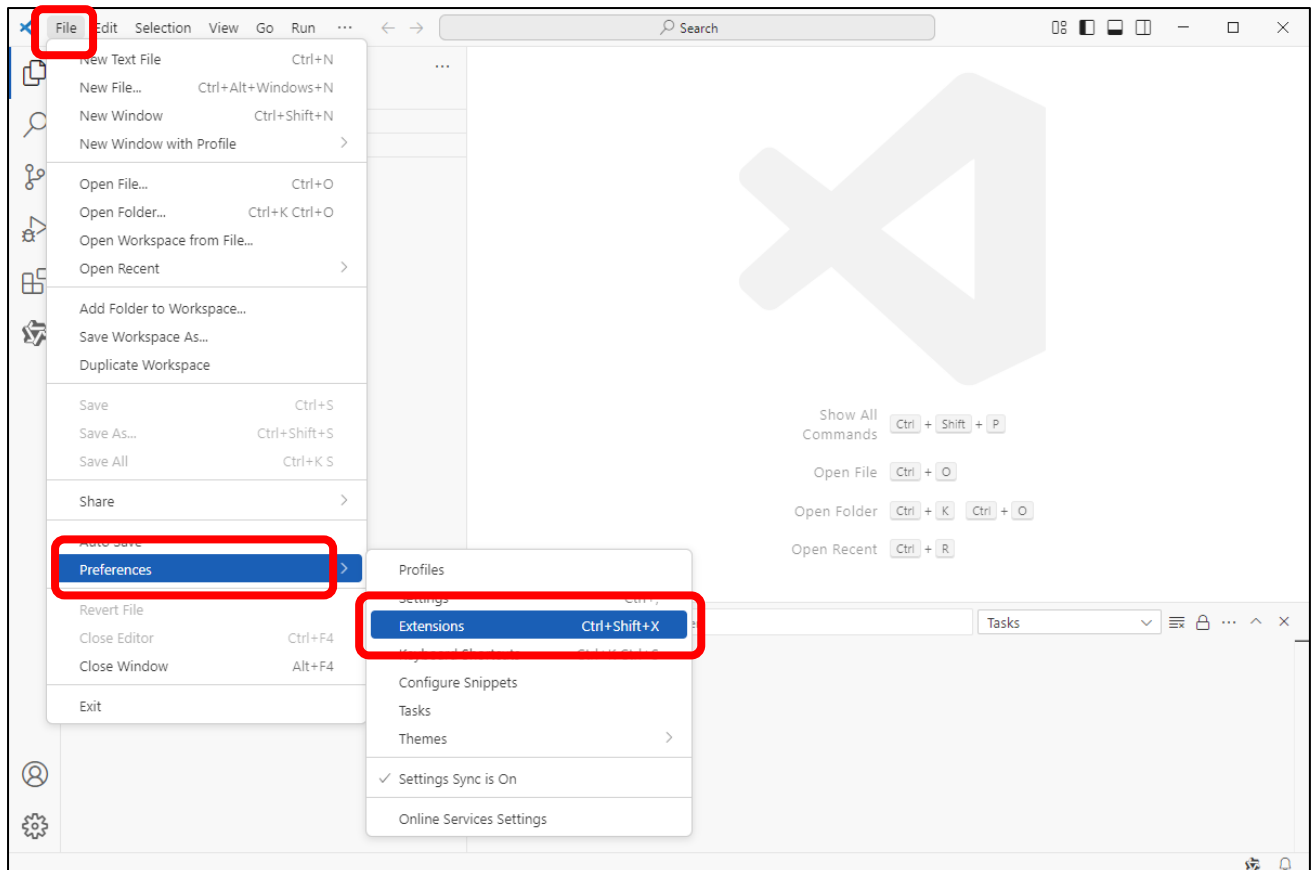


ESP-IDF V5.4.1

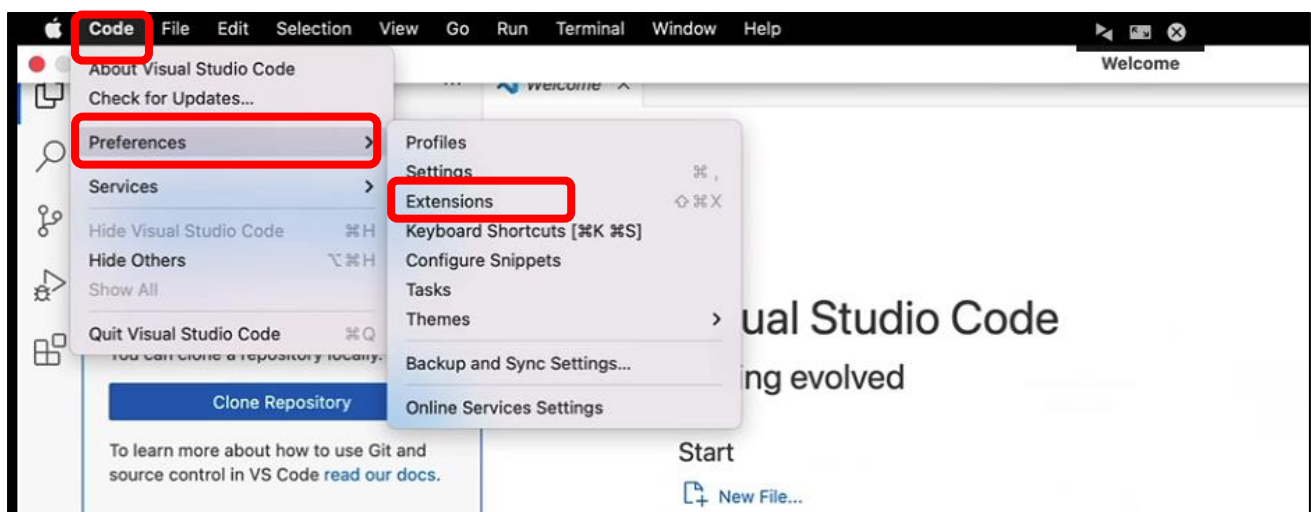
Visual Studio Code is a versatile code editor. To program with the ESP-IDF SDK, we need to install the ESP-IDF extension for it.

Open Visual Studio Code.

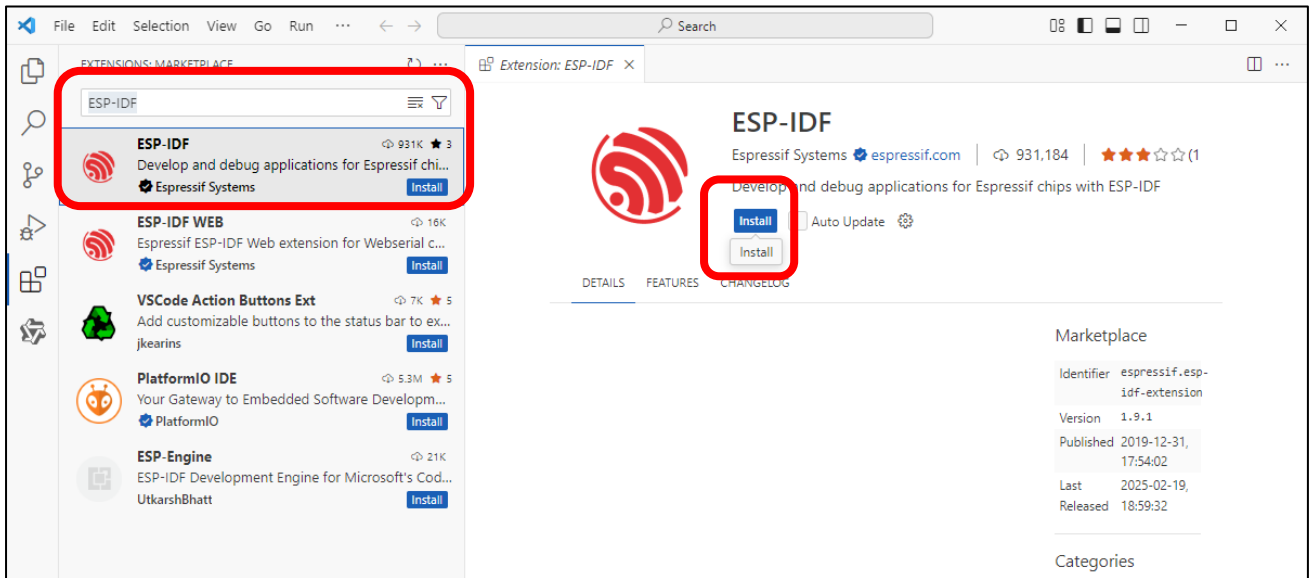
Click on the menu bar: File -> Preferences -> Extensions.



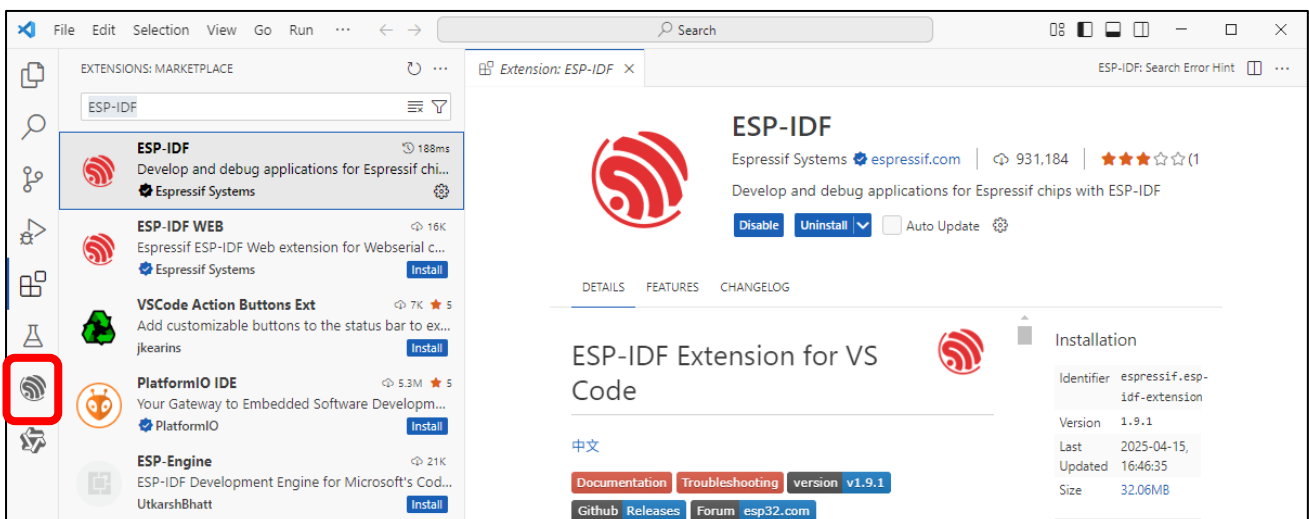
Mac OS: Click "Code" -> "Preferences" -> "Extensions" on the menu bar.



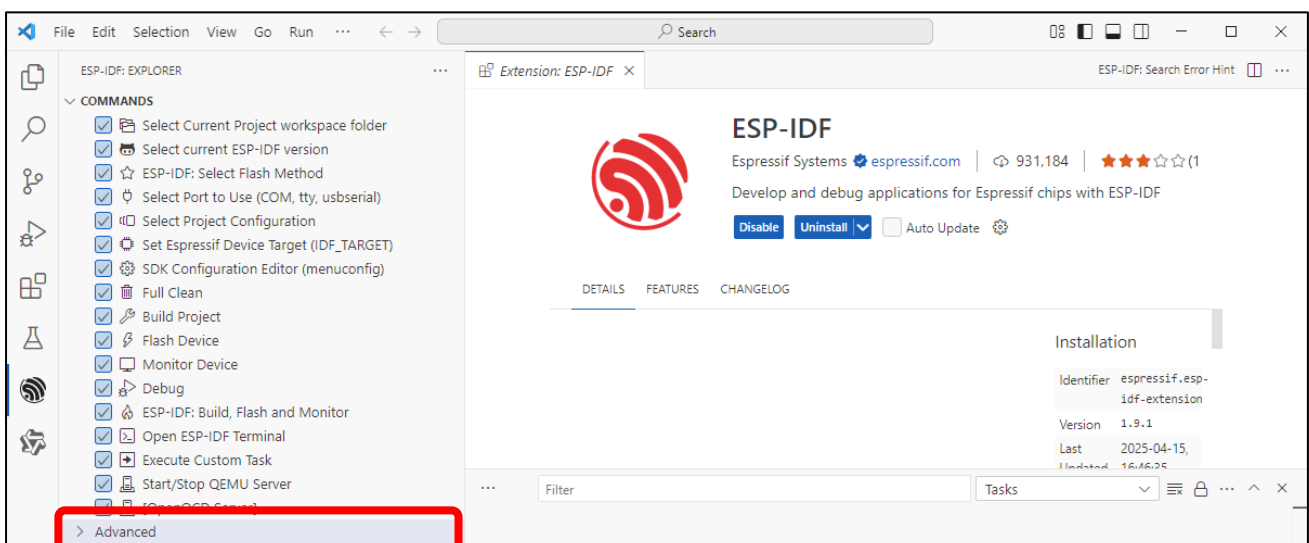
Search for "ESP-IDF" in the extension bar, select the correct result from the list, then click the Install button to proceed.



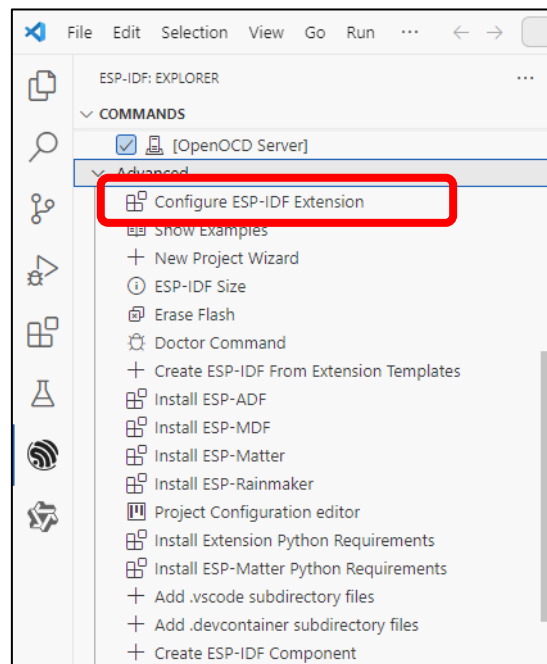
The ESP-IDF extension icon will now appear in the left sidebar - click it to continue.



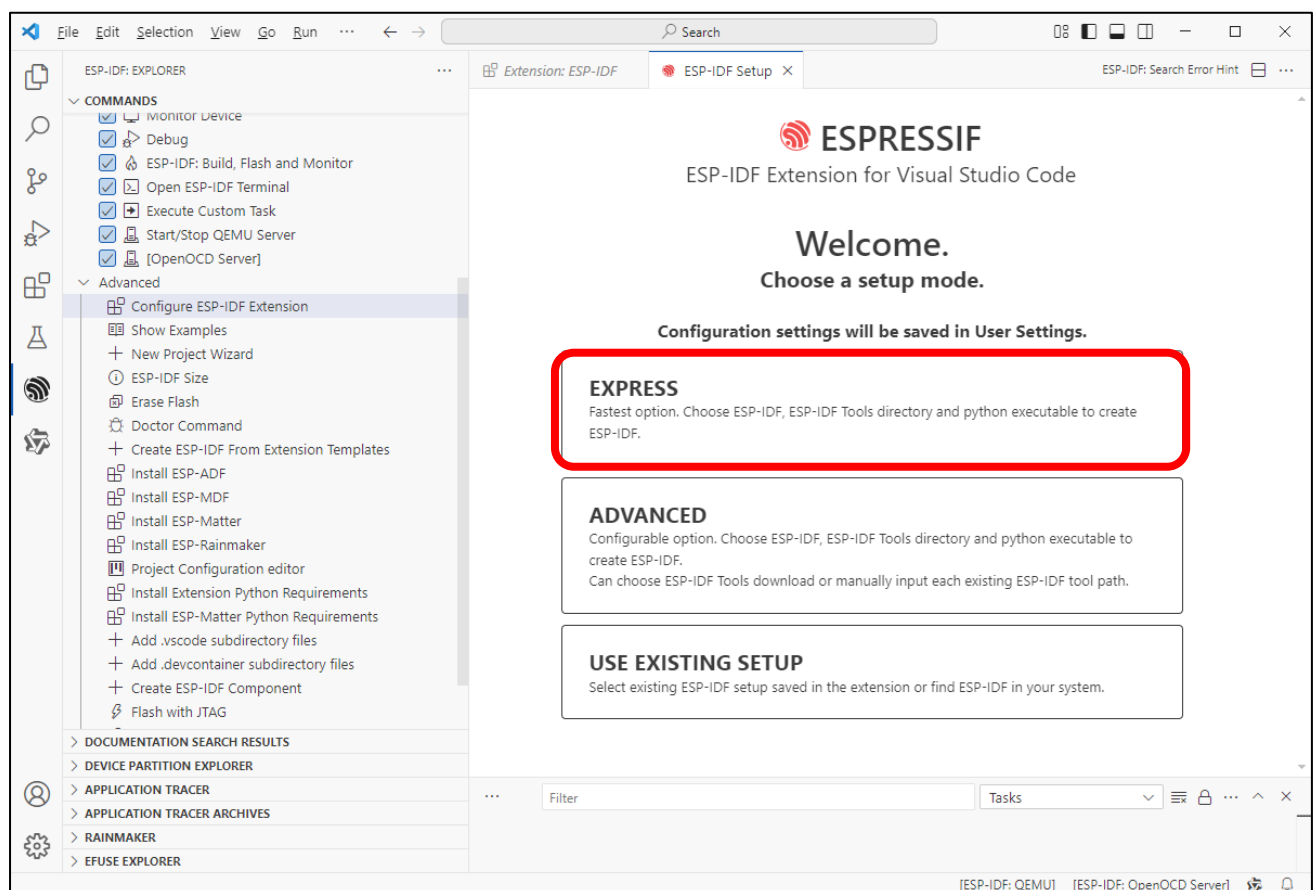
Scroll down with your mouse, locate and click on the "Advanced" option.



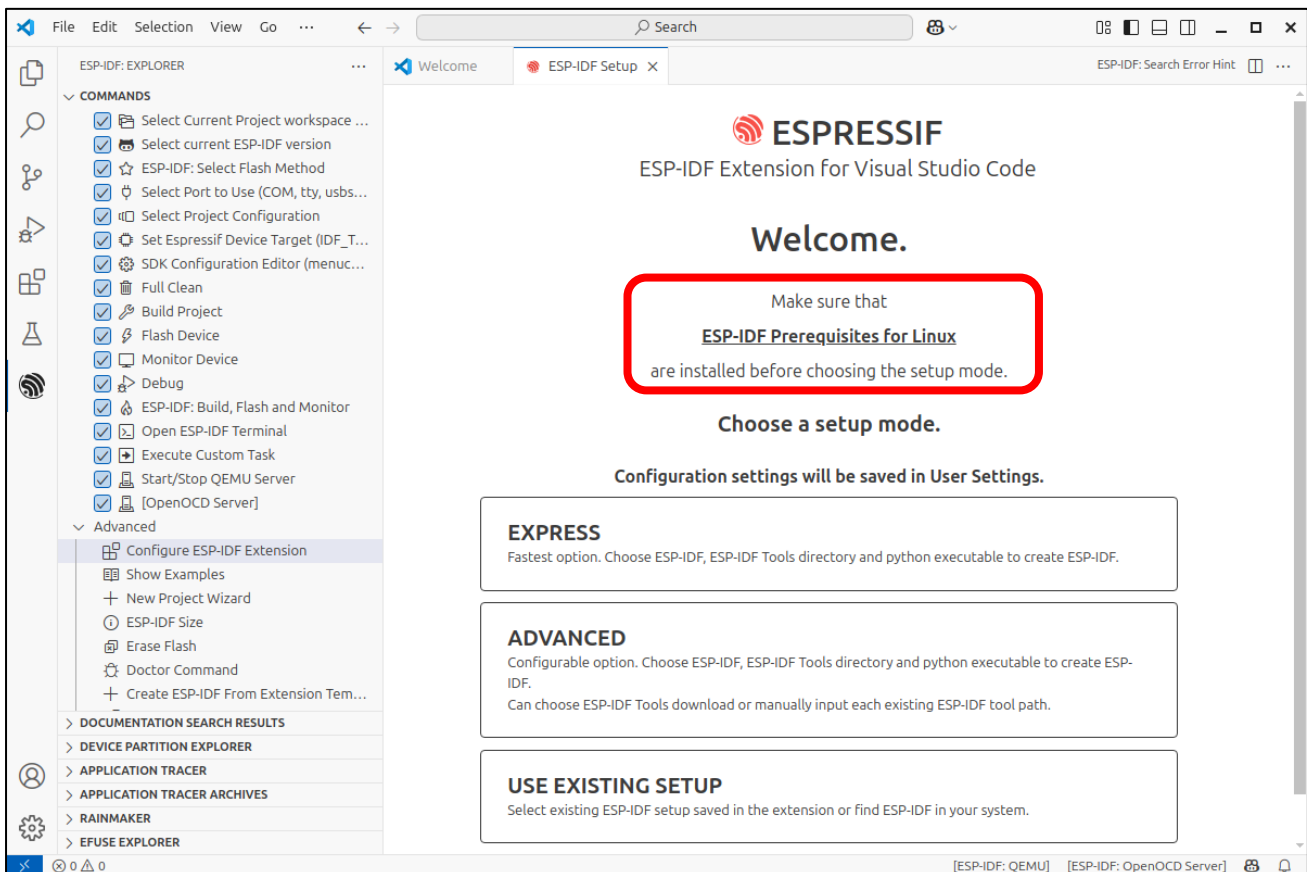
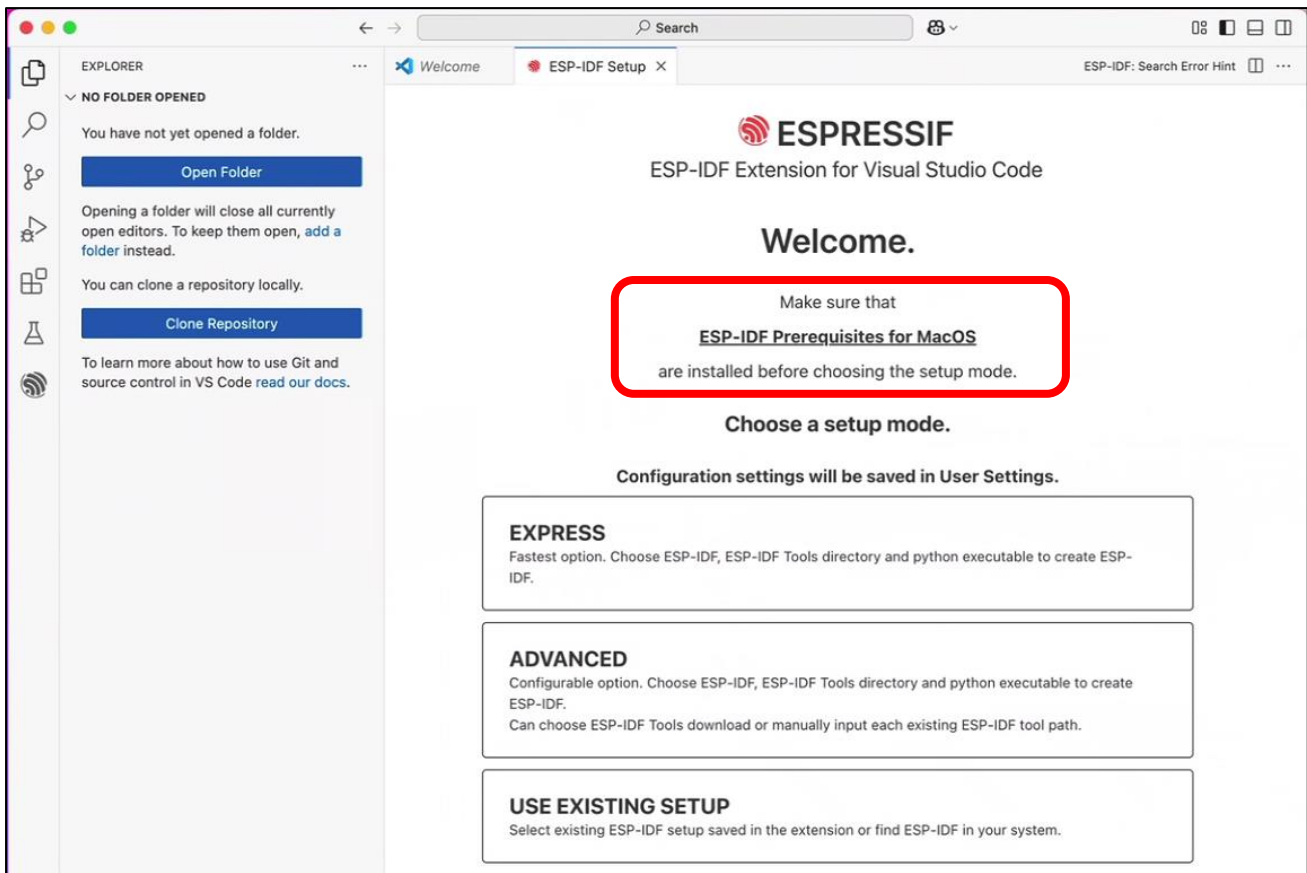
Click the first option: "Configure ESP-IDF Extension".



Select "EXPRESS" on the right.

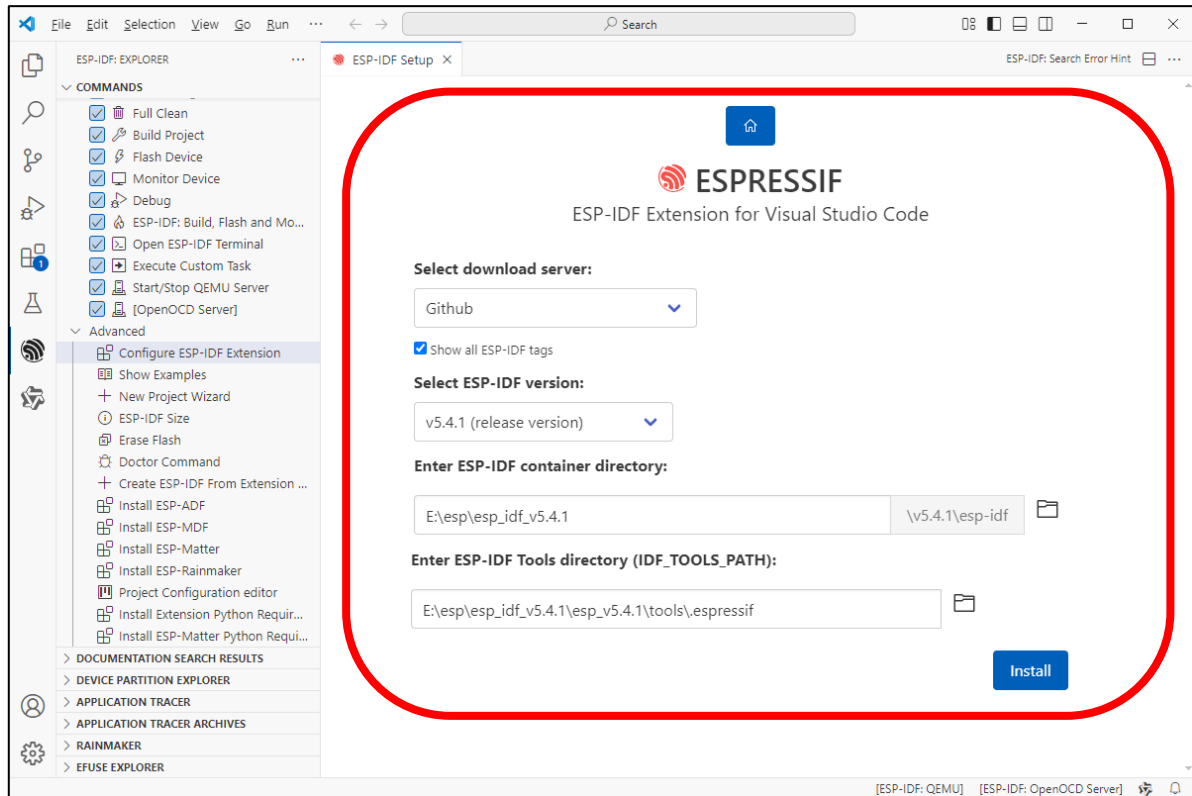


Note: If you're using macOS or Ubuntu, please complete the necessary preparations as prompted before proceeding with installation.



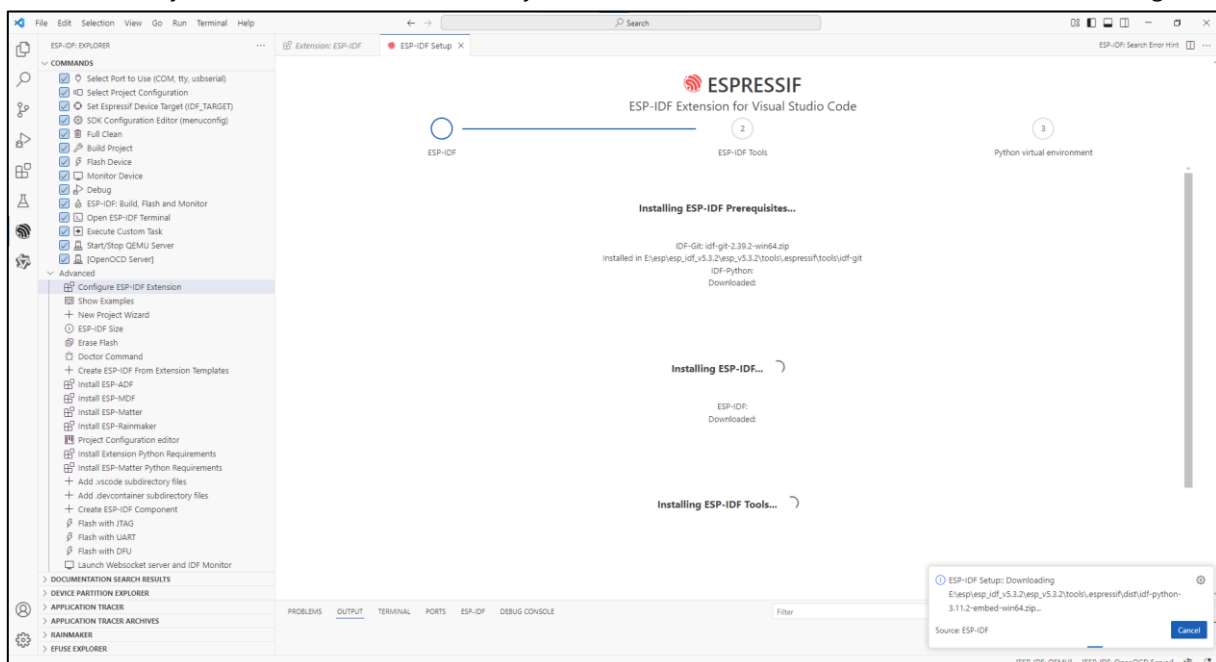
1. Check the box for "Show all ESP-IDF tags"
2. Select "v5.4.1 (release version)" from the dropdown
3. Choose your desired installation path for the ESP-IDF environment
4. Click "Install" to begin the setup

The installation path varies among computer systems, please remember it.



The process will complete automatically.

If it failed, locate your chosen ESP-IDF directory, remove the failed installation folder and install it again.



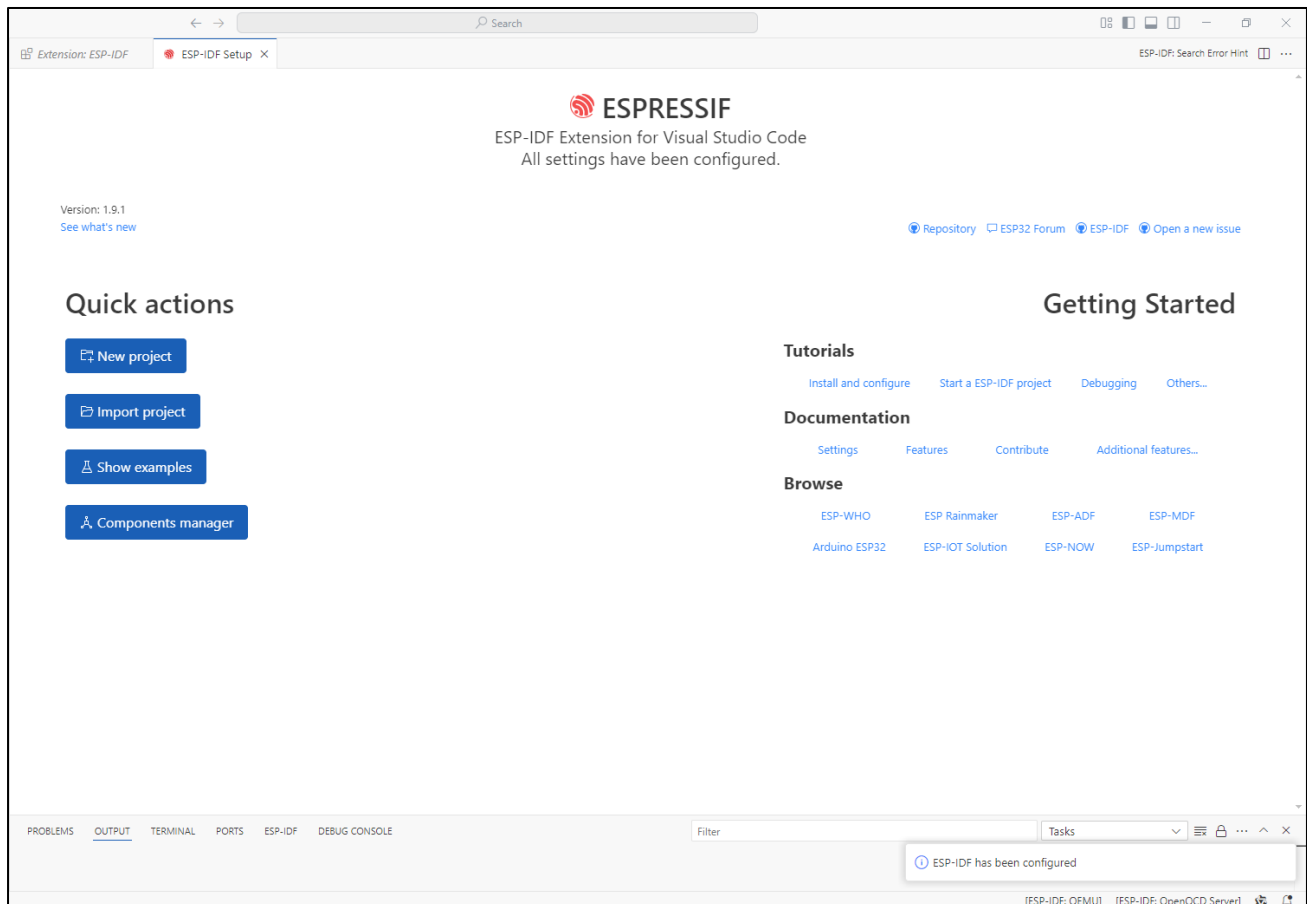
This step may take a while, so please ensure you have a stable and fast internet connection.

If the installation continues to fail, check the relevant link for your operating system below:

Window: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/windows-setup.html>

Mac & Linux: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/linux-macos-setup.html>

The complete installation is as shown below.



For more about ESP-IDF, please refer to

<https://docs.espressif.com/projects/vscode-esp-idf-extension/en/latest/installation.html>

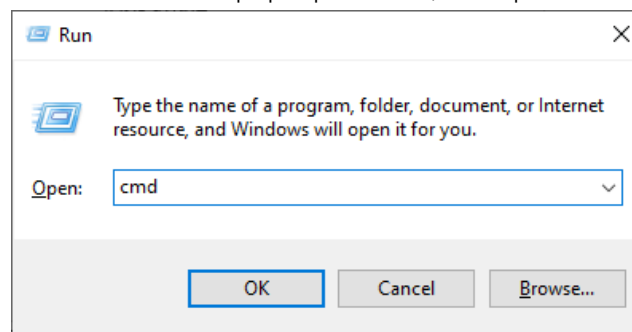
OpenAI Code

This project is derived from the open-source repository: <https://github.com/openai/openai-realtime-embedded>, licensed under MIT License. We have only adapted it for third-party learning and AI functionality trials, without any commercial promotion or application. This tutorial is intended solely for enthusiasts to supplement their learning.

Code Downloading

Windows

Use the shortcut "Win+R", enter "CMD" in the pop-up window, and open the CMD interface.



In the Terminal, install the code with git command.

```
git clone --recurse-submodules https://github.com/Freenove/openai-realtime-embedded
```

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DESKTOP-LIN>git clone --recurse-submodules https://github.com/Freenove/openai-realtime-embedded_
```

The installation is completed as shown below.

```
remote: Enumerating objects: 3984, done.
remote: Counting objects: 100% (1409/1409), done.
remote: Compressing objects: 100% (351/351), done.
remote: Total 3984 (delta 1259), reused 1065 (delta 1056), pack-reused 2575 (from 4)
Receiving objects: 100% (3984/3984), 7.46 MiB | 7.63 MiB/s, done.
Resolving deltas: 100% (2467/2467), done.
Skipping submodule 'deps/libpeer/third_party/coreHTTP/test/unit-test/CMock'
Submodule path 'deps/libpeer/third_party/coreHTTP/source/dependency/3rdparty/11http': checked out 'a4aa7a70e8b9a67f378b53264c61bb044a224366'
Submodule path 'deps/libpeer/third_party/coreMQTT': checked out '03290fe0274e1c9cee5f3608e197daedbfdd1e1f'
Submodule 'test/unit-test/CMock' (https://github.com/ThrowTheSwitch/CMock) registered for path 'deps/libpeer/third_party/coreMQTT/test/unit-test/CMock'
Skipping submodule 'deps/libpeer/third_party/coreMQTT/test/unit-test/CMock'
Submodule path 'deps/libpeer/third_party/libsrtp': checked out '90d05bf8980d16e4ac3f16c19b77e296c4bc207b'
Submodule path 'deps/libpeer/third_party/mbedtls': checked out '1873d3bfc2da771672bd8e7e8f41f57e0af77f33'
Submodule path 'deps/libpeer/third_party/ursctp': checked out '01cc4e042e2235b29d9d489d89728a6f9ac063ed'
C:\Users\DESKTOP-LIN>
```

If you do not have the git tool on your computer, please download and install it by visiting <https://git-scm.com/downloads>

MAC & Linux

Open the Terminal, run the following command, and wait for the installation to finish.

```
git clone --recurse-submodules https://github.com/Freenove/openai-realtime-embedded
```



```
freenove@PandeMacBook-Air ~ % git clone --recurse-submodules https://github.com/Freenove/openai-realtime-embedded
Cloning into 'openai-realtime-embedded'...
remote: Enumerating objects: 72, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (33/33), done.
remote: Total 72 (delta 22), reused 18 (delta 13), pack-reused 26 (from 2)
Receiving objects: 100% (72/72), 18.66 KiB | 1.55 MiB/s, done.
Resolving deltas: 100% (23/23), done.
Submodule 'components/esp-libopus' (https://github.com/XasWorks/esp-libopus.git) registered for path 'components/esp-libopus'
Submodule 'components/esp-protocols' (https://github.com/espressif/esp-protocols.git) registered for path 'components/esp-protocols'
Submodule 'components/srtp' (https://git@github.com/sepfy/esp_ports) registered for path 'components/srtp'
Submodule 'deps/libpeer' (https://github.com/sean-der/libpeer) registered for path 'deps/libpeer'
Cloning into '/Users/freenove/openai-realtime-embedded/components/esp-libopus'...
remote: Enumerating objects: 295, done.
remote: Counting objects: 100% (295/295), done.
remote: Compressing objects: 100% (206/206), done.
remote: Total 295 (delta 90), reused 293 (delta 88), pack-reused 0 (from 0)
Receiving objects: 100% (295/295), 679.98 KiB | 1.98 MiB/s, done.
Resolving deltas: 100% (90/90), done.
Cloning into '/Users/freenove/openai-realtime-embedded/components/esp-protocols'...
remote: Enumerating objects: 18172, done.
remote: Counting objects: 100% (681/681), done.
```

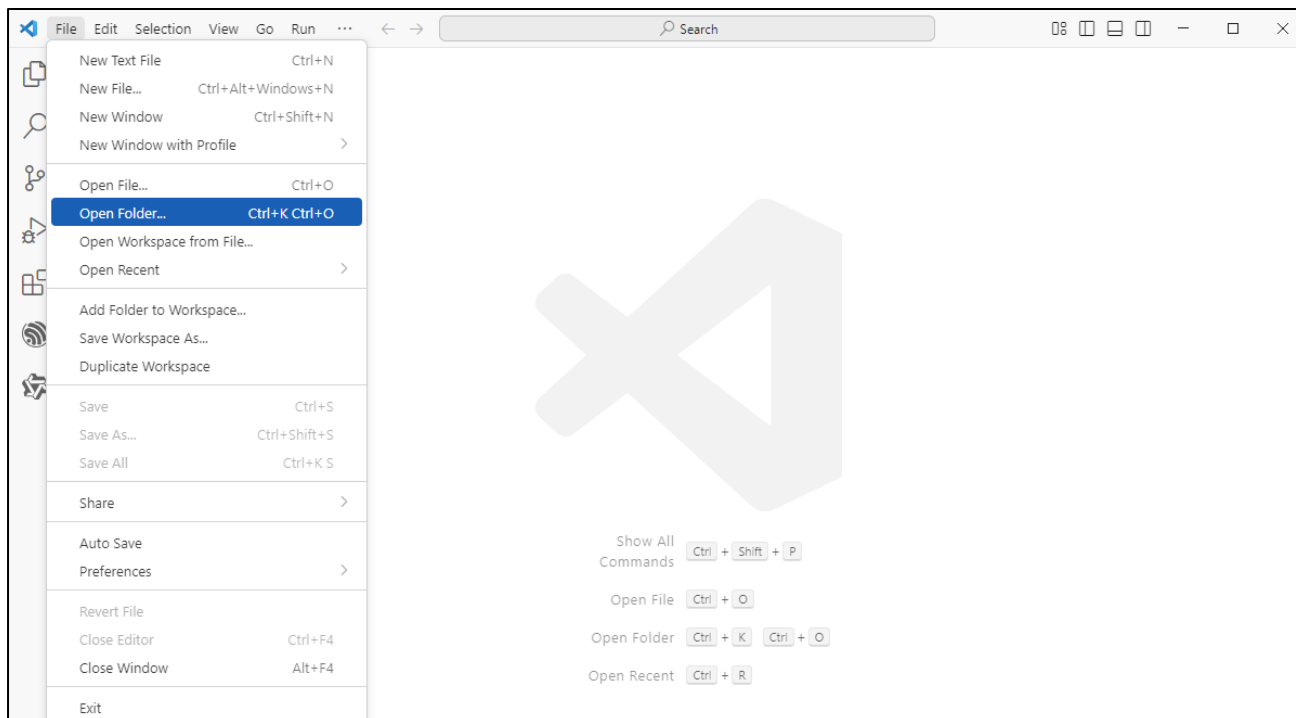
The installation completes as shown below.



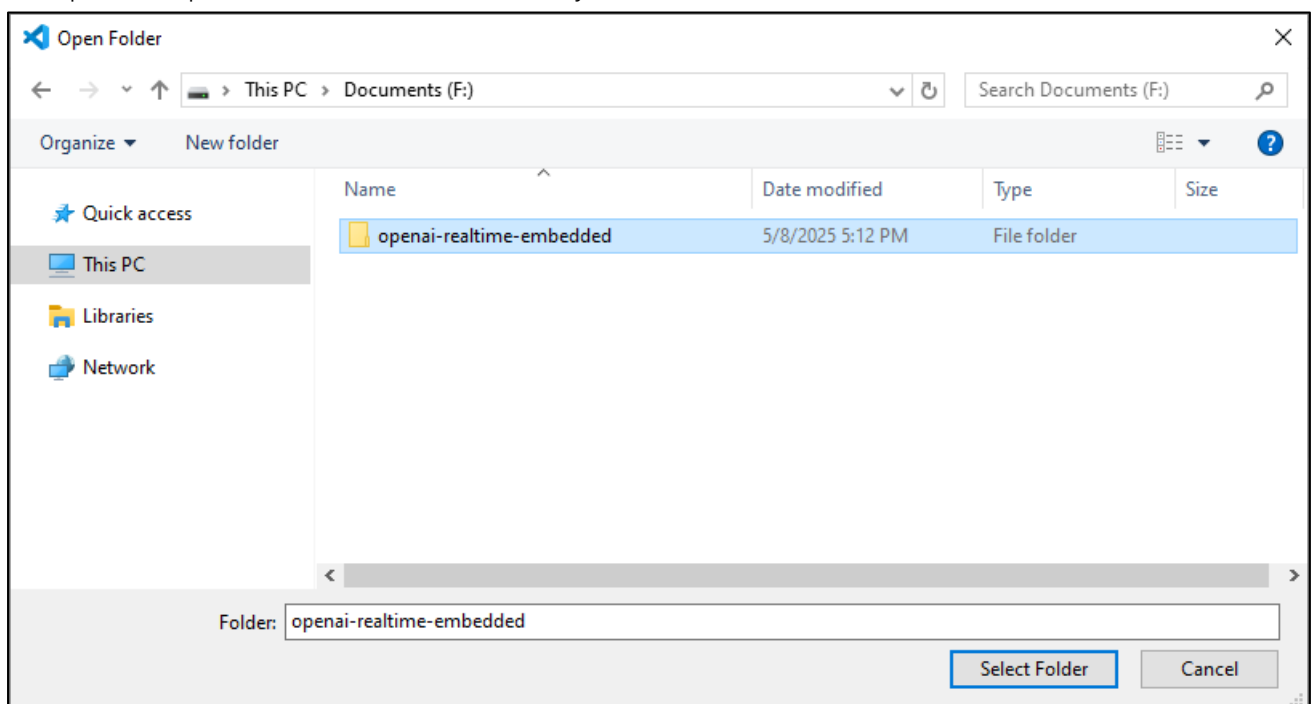
```
Submodule path 'deps/libpeer/third_party/coreHTTP': checked out 'b539e7ab2360efde3c6361f4c2bfcc065b22d087'
Submodule 'source/dependency/3rdparty/llhttp' (https://github.com/nodejs/llhttp.git) registered for path 'deps/libpeer/third_party/coreHTTP/source/dependency/3rdparty/llhttp'
Submodule 'test/unit-test/CMock' (https://github.com/ThrowTheSwitch/CMock.git) registered for path 'deps/libpeer/third_party/coreHTTP/test/unit-test/CMock'
Cloning into '/Users/freenove/openai-realtime-embedded/deps/libpeer/third_party/coreHTTP/source/dependency/3rdparty/llhttp'...
..
remote: Enumerating objects: 3984, done.
remote: Counting objects: 100% (1410/1410), done.
remote: Compressing objects: 100% (351/351), done.
remote: Total 3984 (delta 1260), reused 1066 (delta 1057), pack-reused 2574 (from 4)
Receiving objects: 100% (3984/3984), 7.46 MiB | 6.37 MiB/s, done.
Resolving deltas: 100% (2467/2467), done.
Skipping submodule 'deps/libpeer/third_party/coreHTTP/test/unit-test/CMock'
Submodule path 'deps/libpeer/third_party/coreHTTP/source/dependency/3rdparty/llhttp': checked out 'a4aa7a70e8b9a67f378b53264c61bb044a224366'
Submodule path 'deps/libpeer/third_party/coreMQTT': checked out '03290fe0274e1c9cee5f3608e197daedbfdd1e1f'
Submodule 'test/unit-test/CMock' (https://github.com/ThrowTheSwitch/CMock) registered for path 'deps/libpeer/third_party/coreMQTT/test/unit-test/CMock'
Skipping submodule 'deps/libpeer/third_party/coreMQTT/test/unit-test/CMock'
Submodule path 'deps/libpeer/third_party/libsrtp': checked out '90d05bf8980d16e4ac3f16c19b77e296c4bc207b'
Submodule path 'deps/libpeer/third_party/mbedtls': checked out '1873d3bfc2da771672bd8e7e8f41f57e0af77f33'
Submodule path 'deps/libpeer/third_party/usbip': checked out '01cc4e042e2235b29d9d489d89728a6f9ac063ed'
freenove@PandeMacBook-Air ~ %
```

Configuring Code Environment

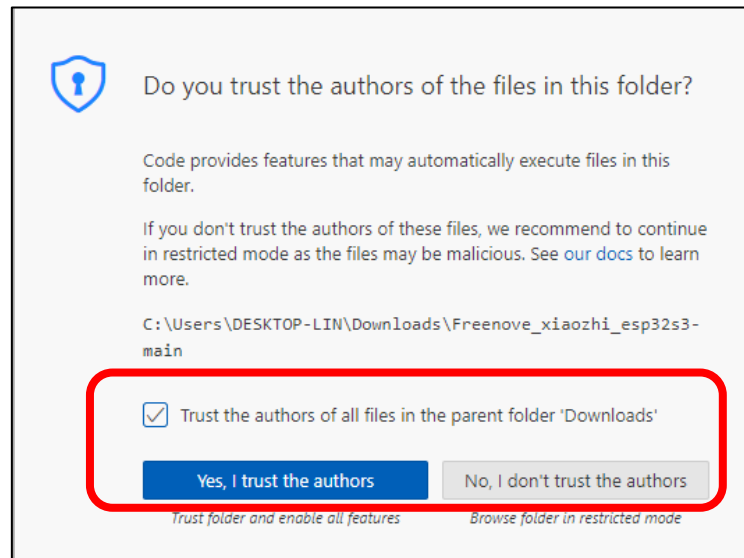
On Visual Studio Code, click “File” -> “Open Folder...”



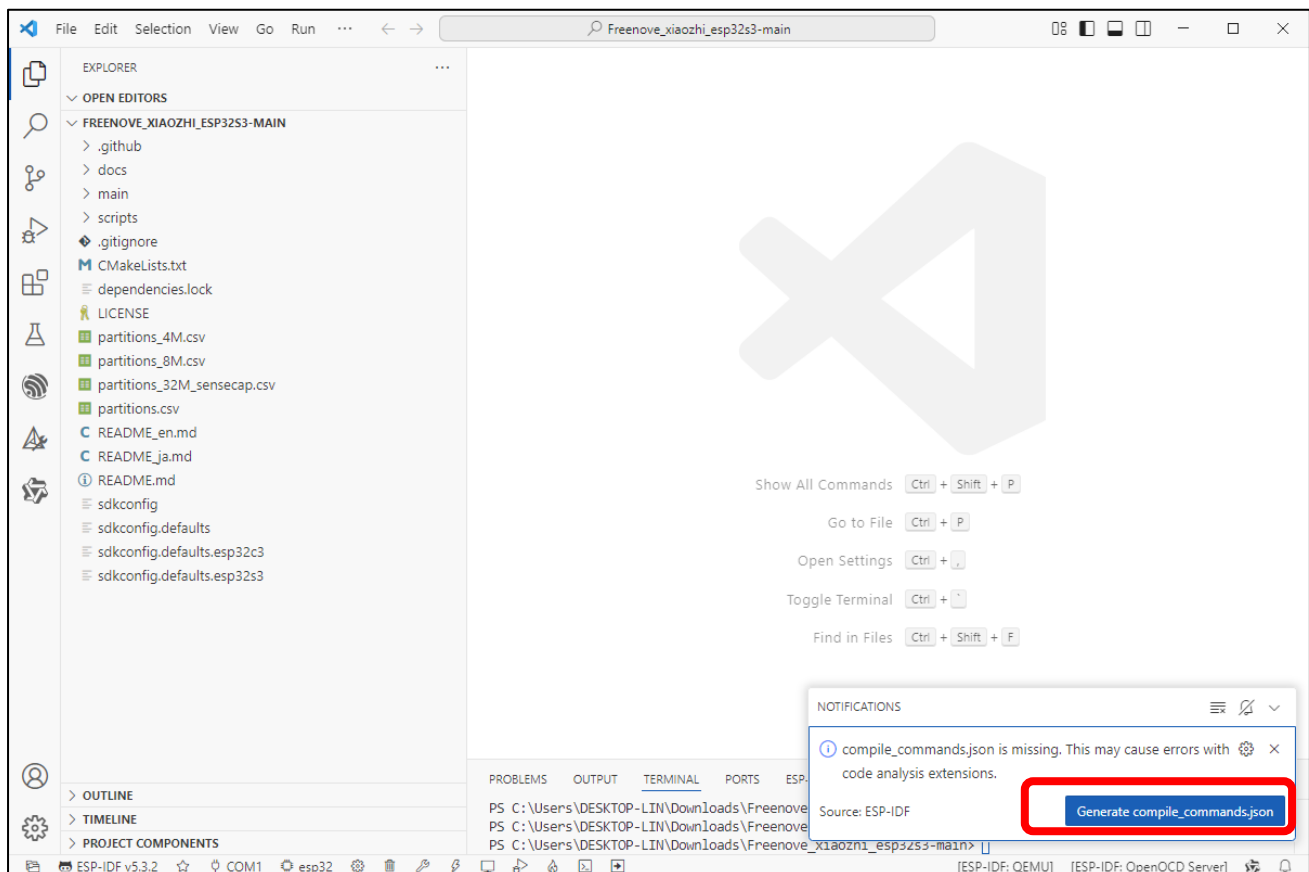
Select the **openai-realtime-embedded** folder. Here, the interface of the Windows system is taken as an example. The operation on the mac and Linux system is similar.



Check the box "Trust the authors of all files in the parent folder 'Downloads'" and select "Yes, I trust the authors".



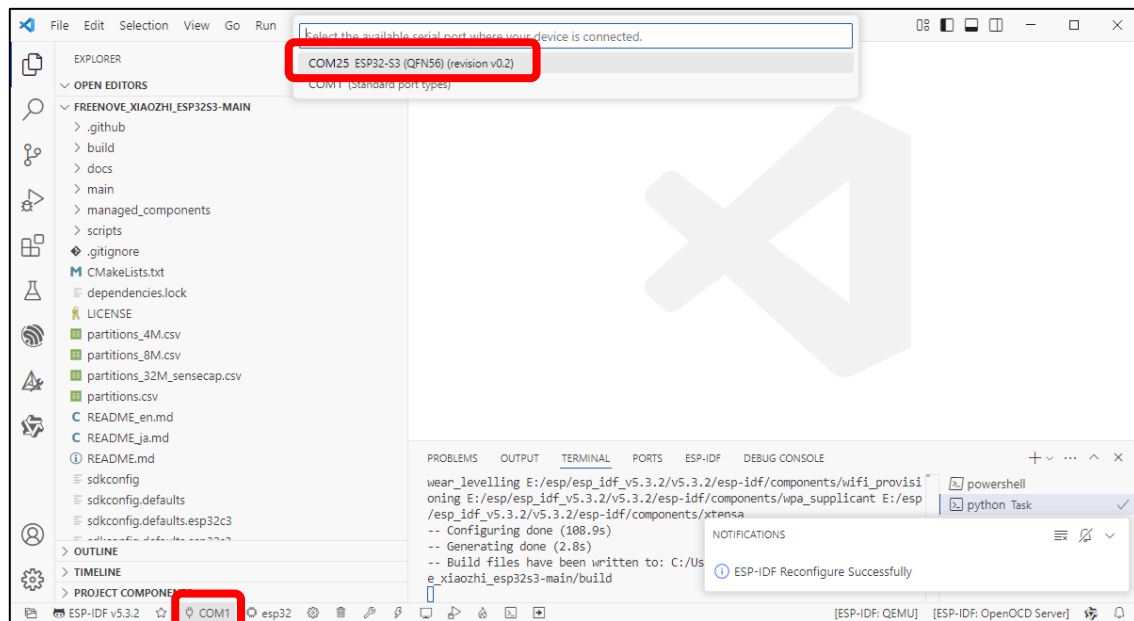
Please note: A pop-up notification 'Generate compile_commands.json' will appear in the lower-right corner. **Please disregard it. DO NOT click it.**



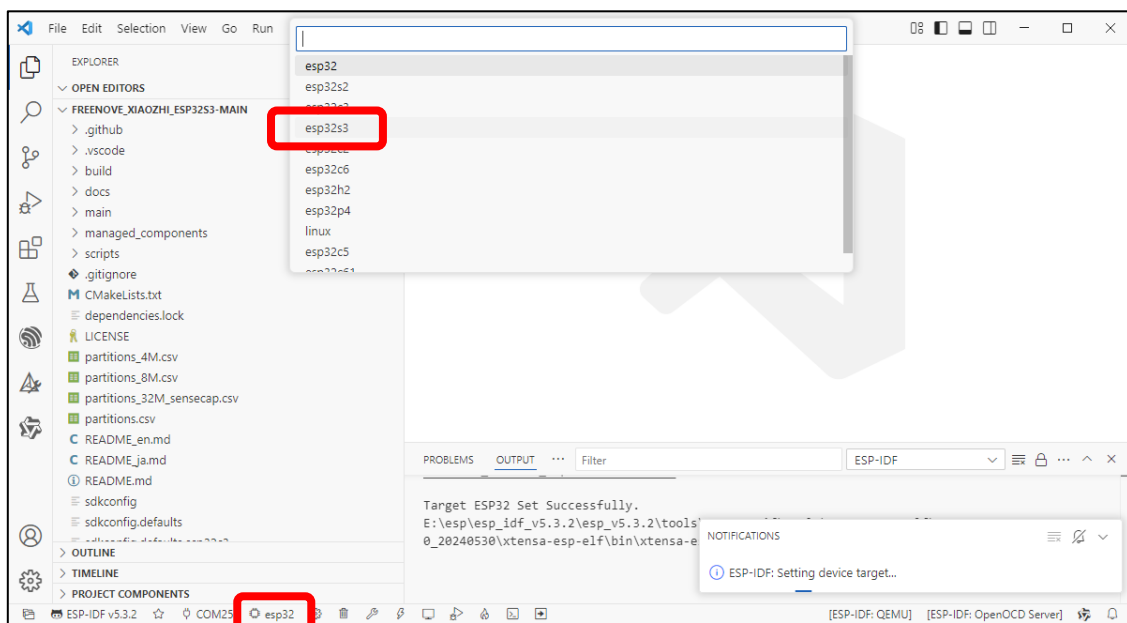
Connect the ESP32S3 WROOM board to your computer with the USB cable (do not use the wrong connector).



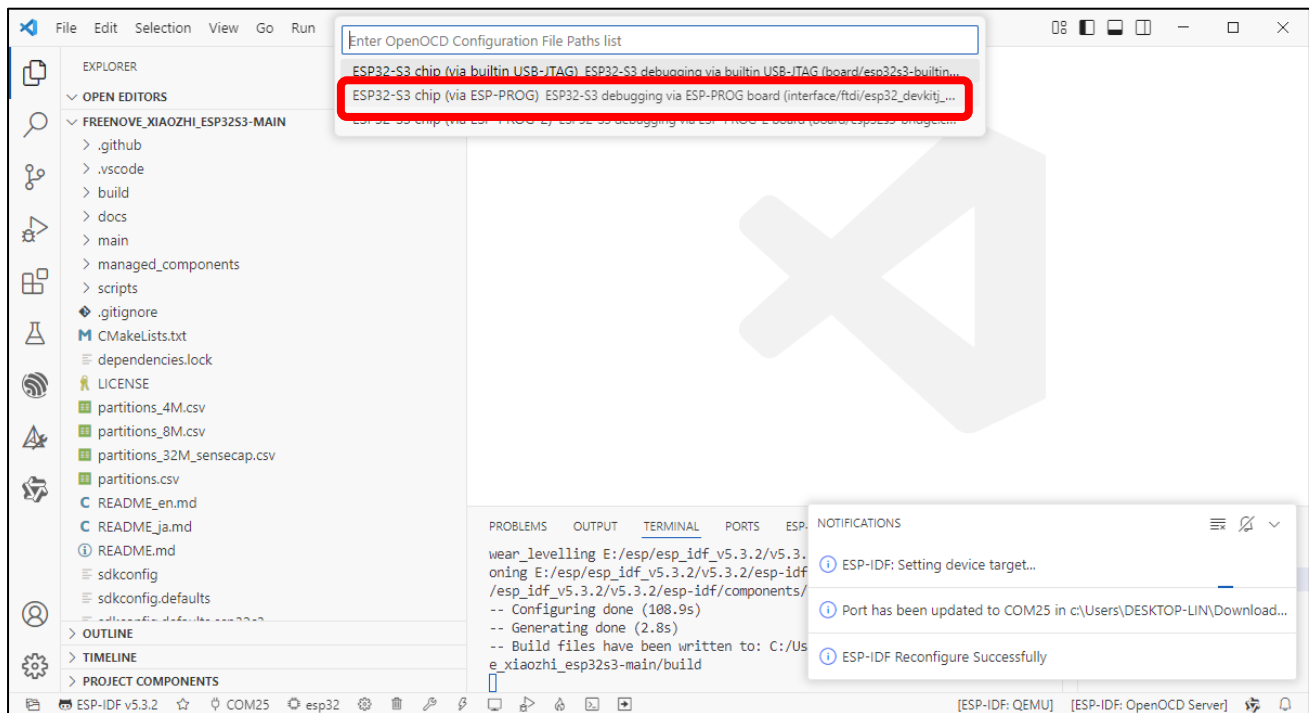
Click on '**COMx**' in the bottom-left corner to display all available COM ports on your computer. Locate and select the entry labeled '**ESP32-S3**'.



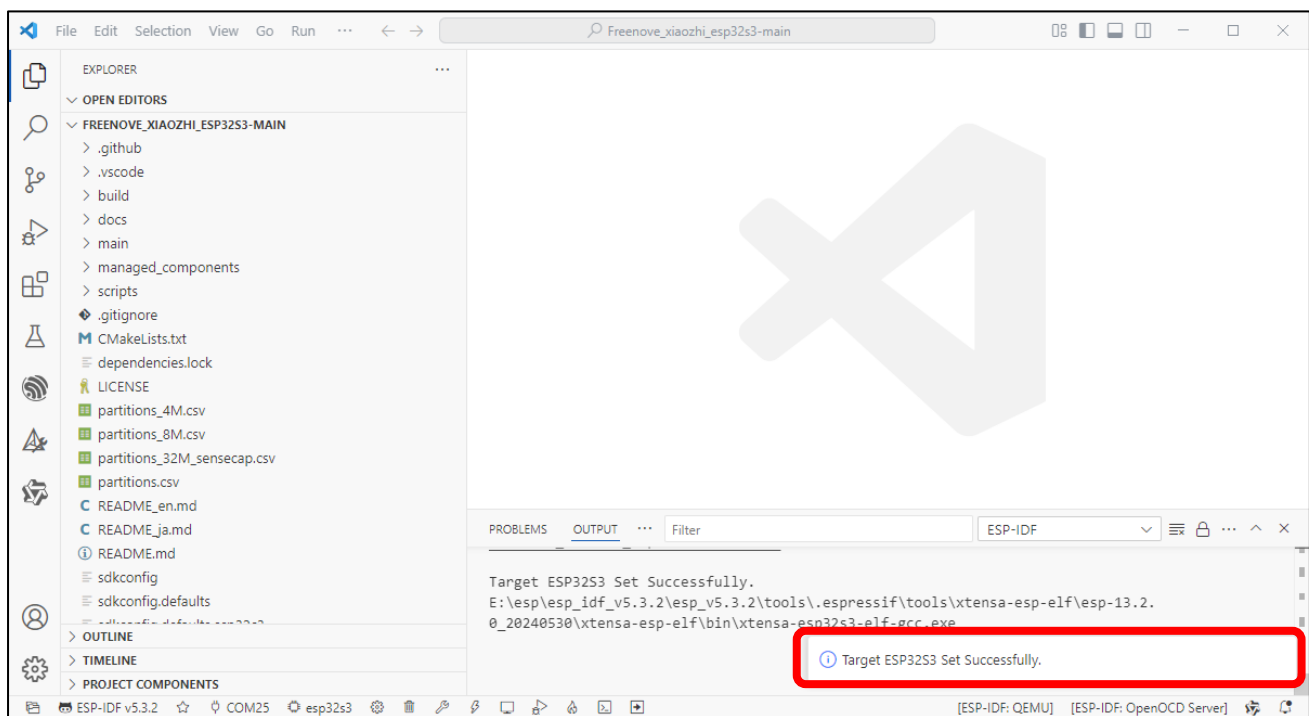
Click the '**ESP32**' button in the bottom-left corner to display all available ESP32 models, then select '**ESP32S3**' from the list."



From the new selection menu, choose 'ESP32-S3 Chip (via ESP-PROG) - ESP32-S3 debugging via ESP-PROG Board...'



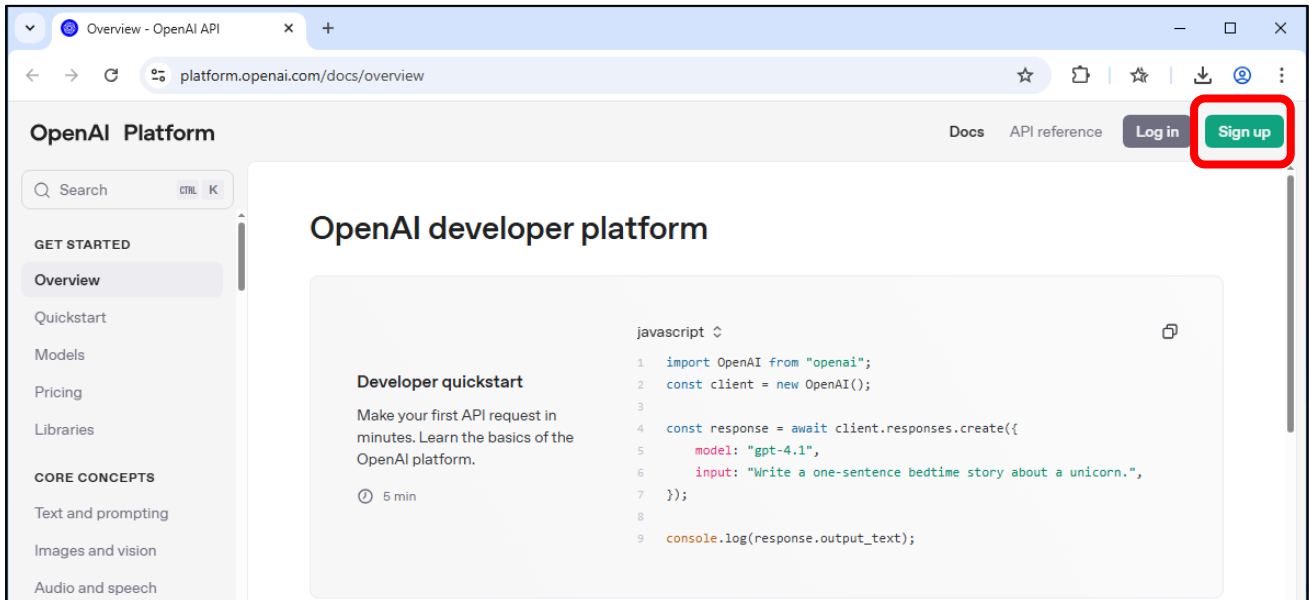
Wait until it shows "Target ESP32S3 Set Successfully" at the bottom right.



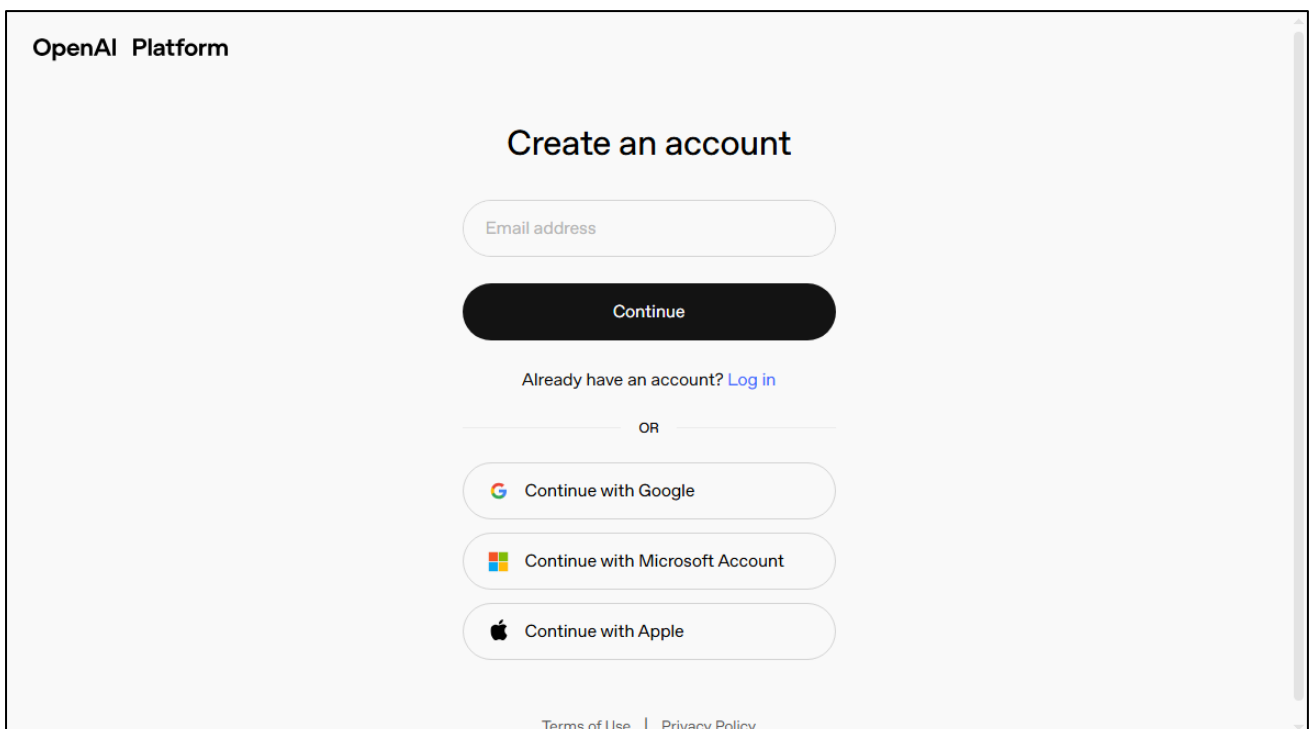
Registering Open API Keys

If you don't have an OpenAI API key yet, please visit <https://platform.openai.com/docs/overview> to sign up and purchase one.

You'll need to create an account. Click "Sign up" in the top-right corner.



Create the account in any way you prefer.



Here we take Email address as an example. Input the email address and password, and click Continue.

OpenAI Platform

Create your account

Set your password for OpenAI to continue

[Redacted]@com [Edit](#)

[Redacted] [\[Eye Icon\]](#)

Your password must contain:

- ✓ [At least 12 characters](#)

[Continue](#)

Already have an account? [Log in](#)

[Terms of Use](#) | [Privacy Policy](#)

OpenAI Platform will send a verification code to your email address. Input the code to the bar, click Continue.

OpenAI Platform

Check your inbox

Enter the verification code we just sent to [Redacted]@com.

[Continue](#)

[Resend email](#)

[Terms of Use](#) | [Privacy Policy](#)

Complete your personal information.

OpenAI Platform

Tell us about you

Full name

Birthday

By clicking "Continue", you agree to our [Terms](#) and have read our [Privacy Policy](#).

Continue

You can either fill the Organization Name or not, then click Create organization.



Welcome to OpenAI Platform

Create an organization to generate API keys and start building.

Organization name

Personal

What best describes you?

Not technical

Create organization

You can click "I'll invite my team later".

OpenAI Platform



Invite your team

Members can make standard API requests and read basic organizational data.

Emails

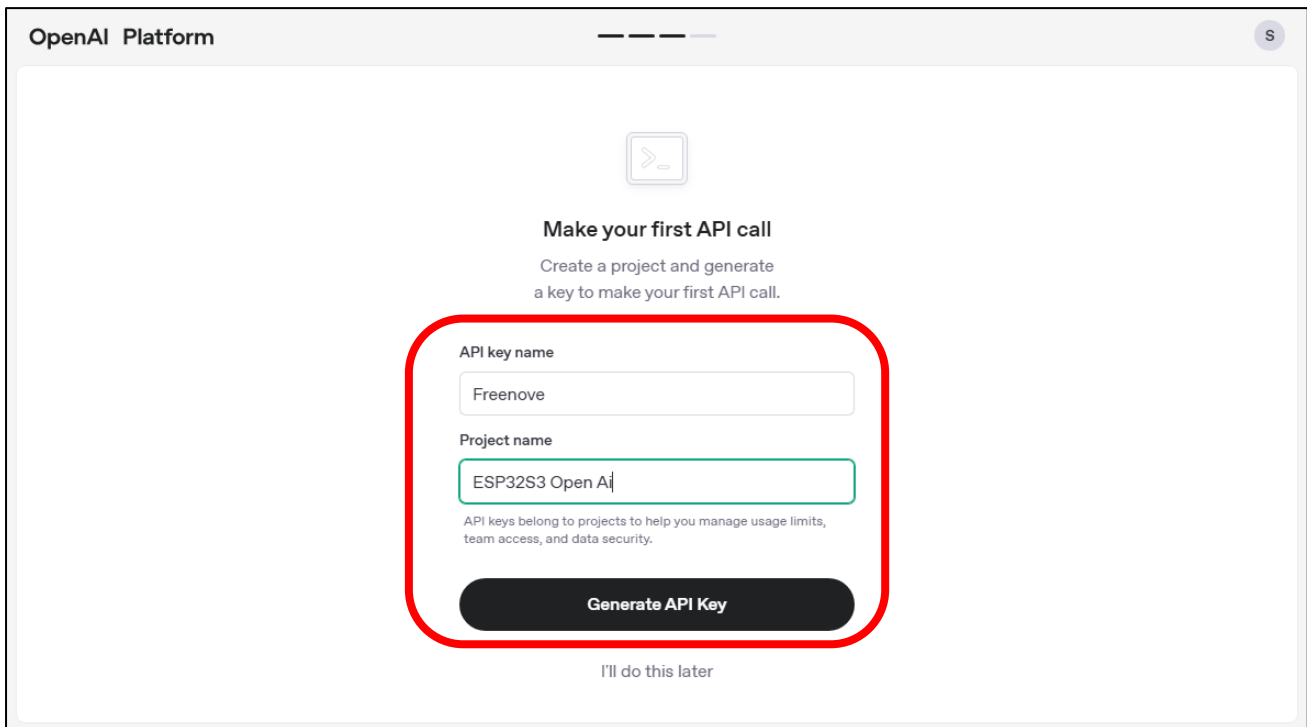
Owner

name@company.com

Continue

I'll invite my team later

Name it whatever you like, and then click "Generate API Key".



OpenAI Platform

Make your first API call

Create a project and generate a key to make your first API call.

API key name

Freenove

Project name

ESP32S3 Open Ai

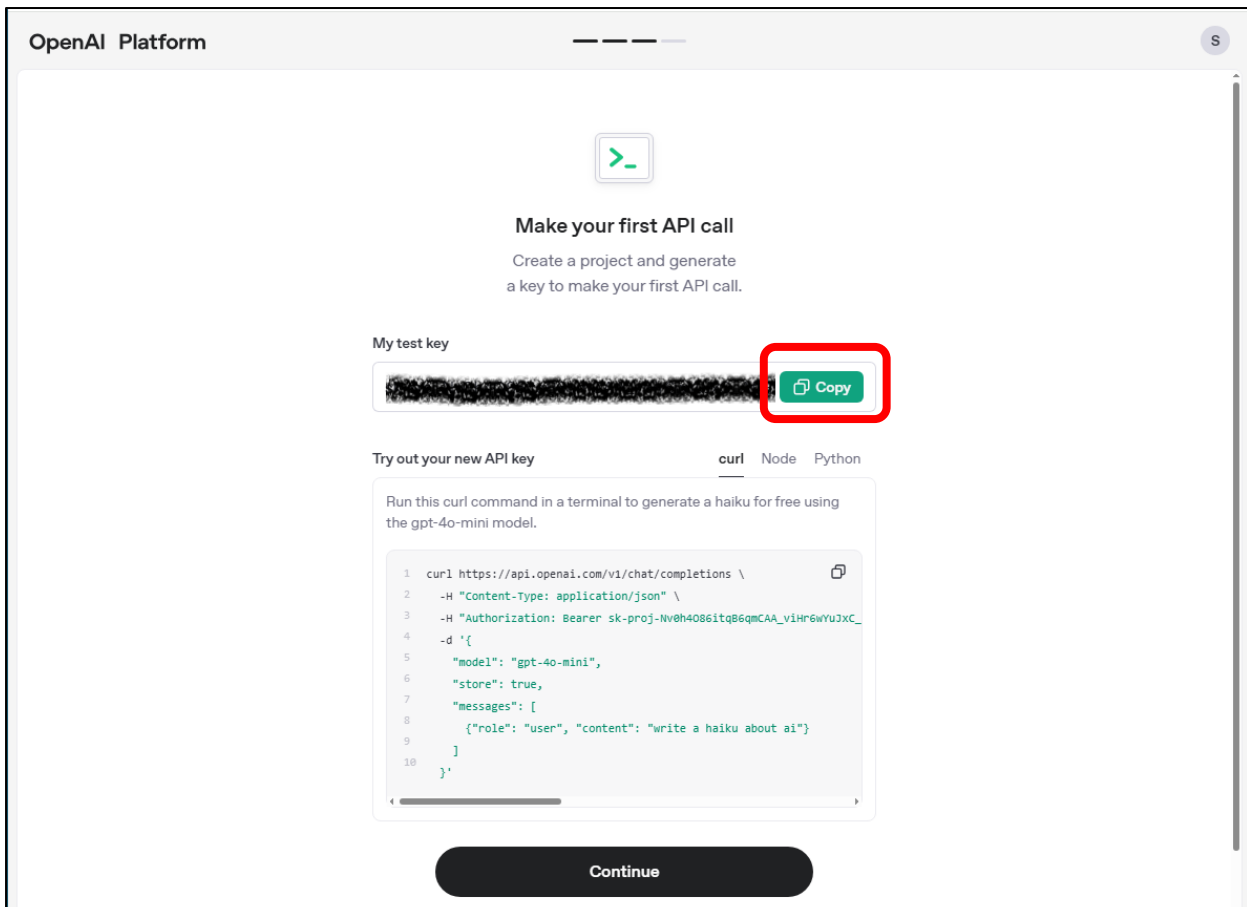
API keys belong to projects to help you manage usage limits, team access, and data security.

Generate API Key

I'll do this later

Click Copy and paste your "My test key" somewhere to save it. If you click "Continue", you will no longer be able to view the key again!

We recommend creating a separate document to store your key securely and prevent loss.



OpenAI Platform

Make your first API call

Create a project and generate a key to make your first API call.

My test key

Copy

Try out your new API key

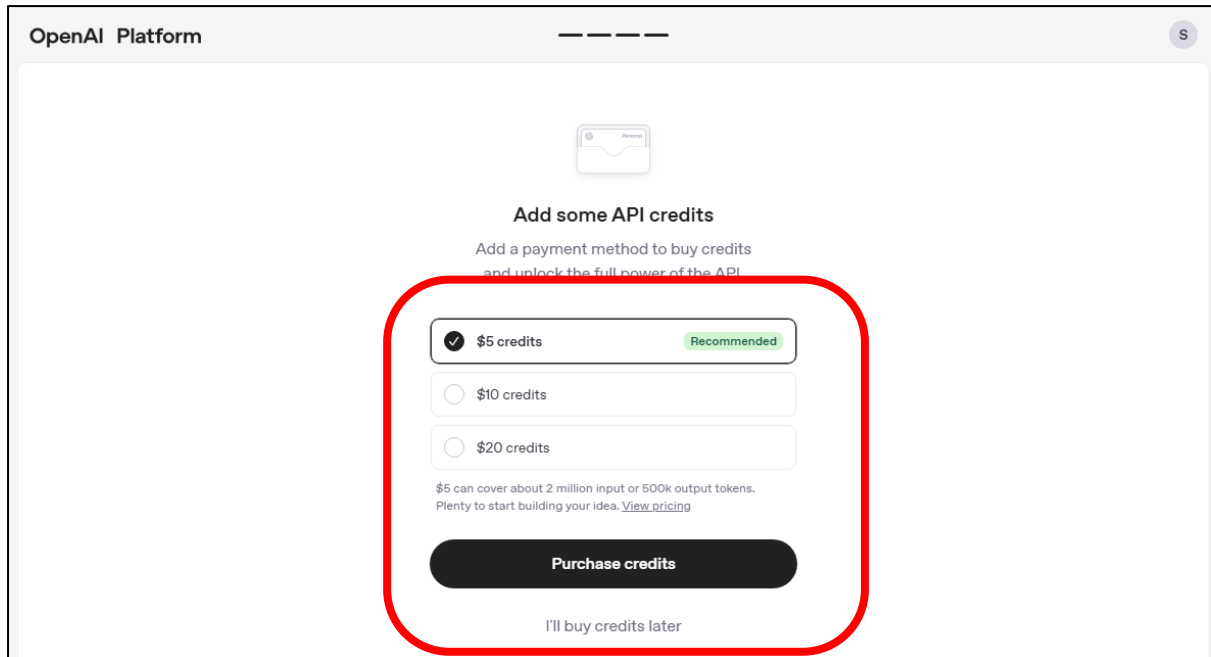
curl Node Python

Run this curl command in a terminal to generate a haiku for free using the gpt-4o-mini model.

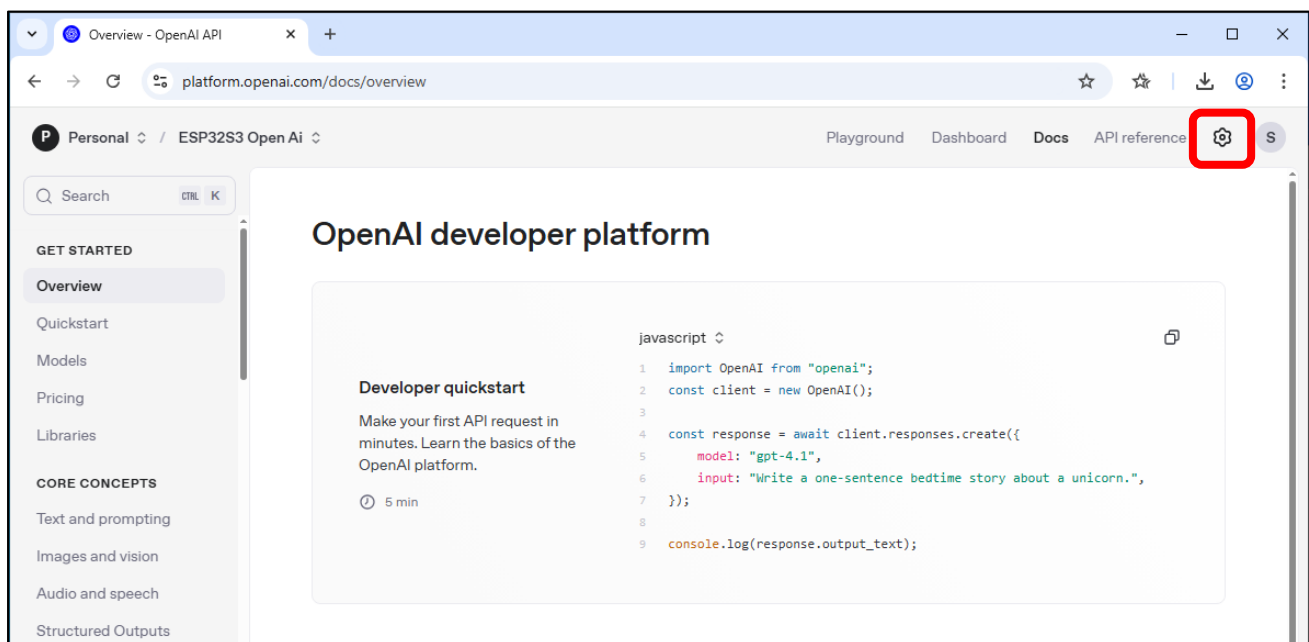
```
1 curl https://api.openai.com/v1/chat/completions \
2 -H "Content-Type: application/json" \
3 -H "Authorization: Bearer sk-proj-Nv0H40861tqB6qmCAA_v1Hr6wYU3XC_
4 -d '{
5   "model": "gpt-4o-mini",
6   "store": true,
7   "messages": [
8     {"role": "user", "content": "write a haiku about ai"}
9   ]
10  }'
```

Continue

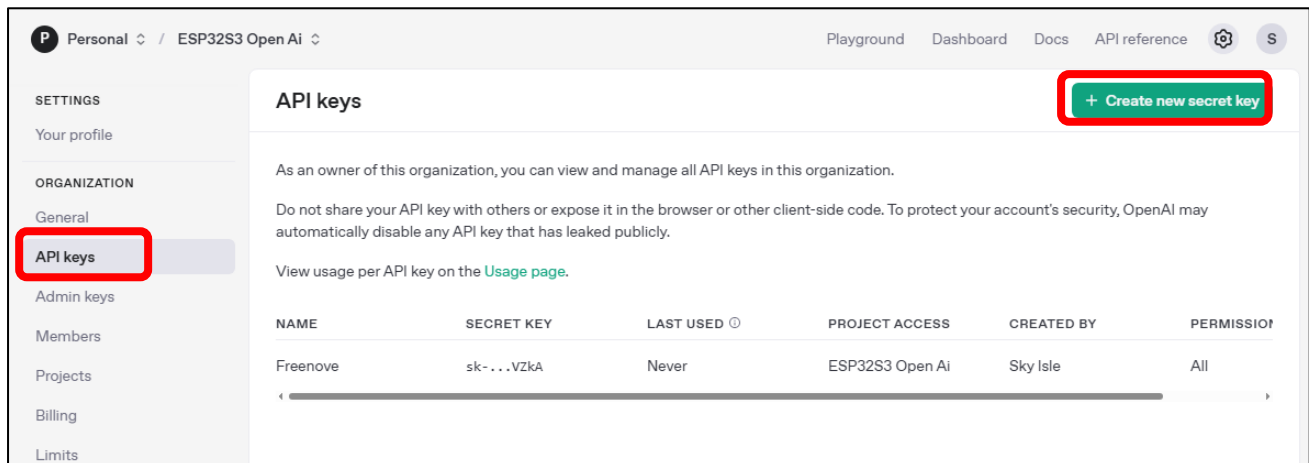
OpenAI operates on a paid service model. You need to purchase for credits to ensure uninterrupted access to its features.



After completion, the interface will appear as shown below. Click the settings icon in the upper right corner.



You can click "API Keys" to manage your OpenAI keys.



API keys

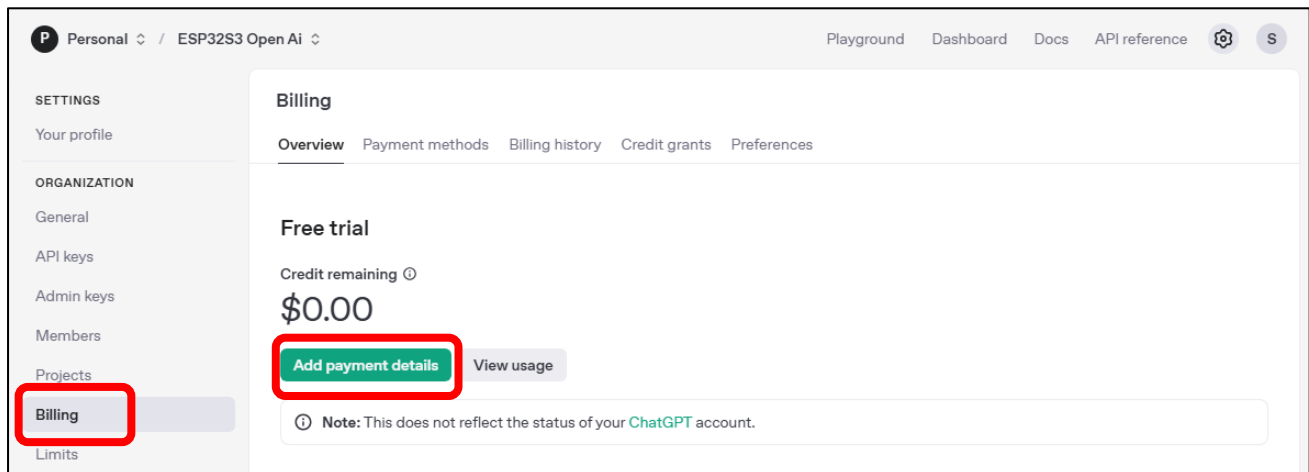
As an owner of this organization, you can view and manage all API keys in this organization.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED ⓘ	PROJECT ACCESS	CREATED BY	PERMISSION
Freenove	sk-...VZkA	Never	ESP32S3 Open Ai	Sky Isle	All

Click "Billing" to check your available balance, or click "Add Payment Details" to top up funds.



Billing

Overview Payment methods Billing history Credit grants Preferences

Free trial

Credit remaining ⓘ

\$0.00

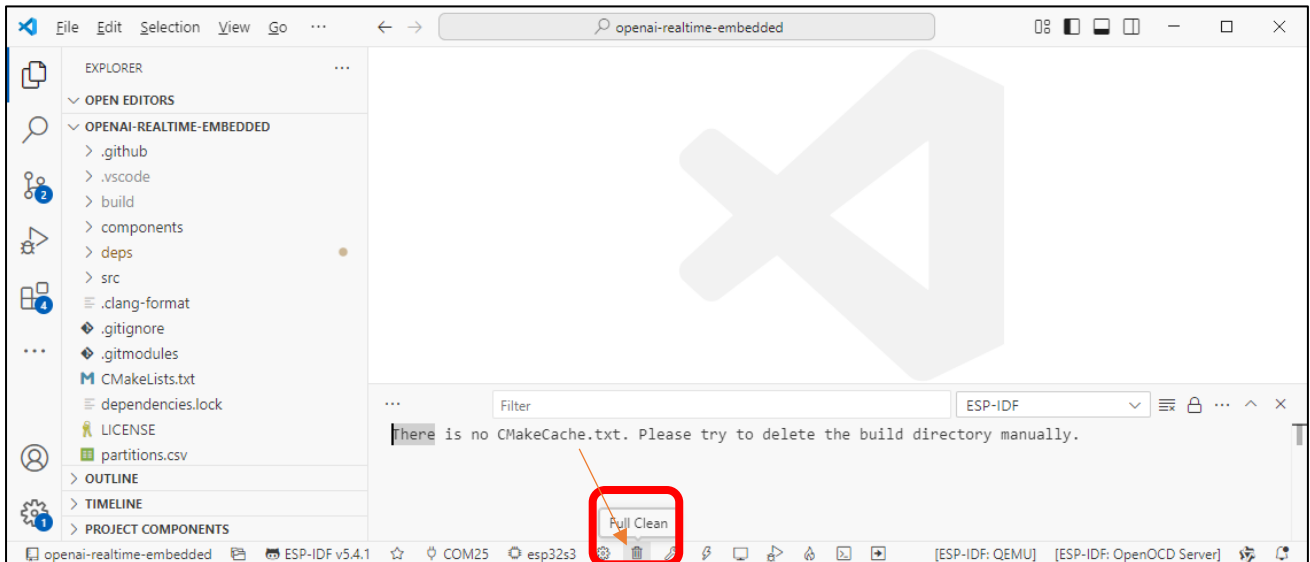
[Add payment details](#) [View usage](#)

ⓘ **Note:** This does not reflect the status of your [ChatGPT](#) account.

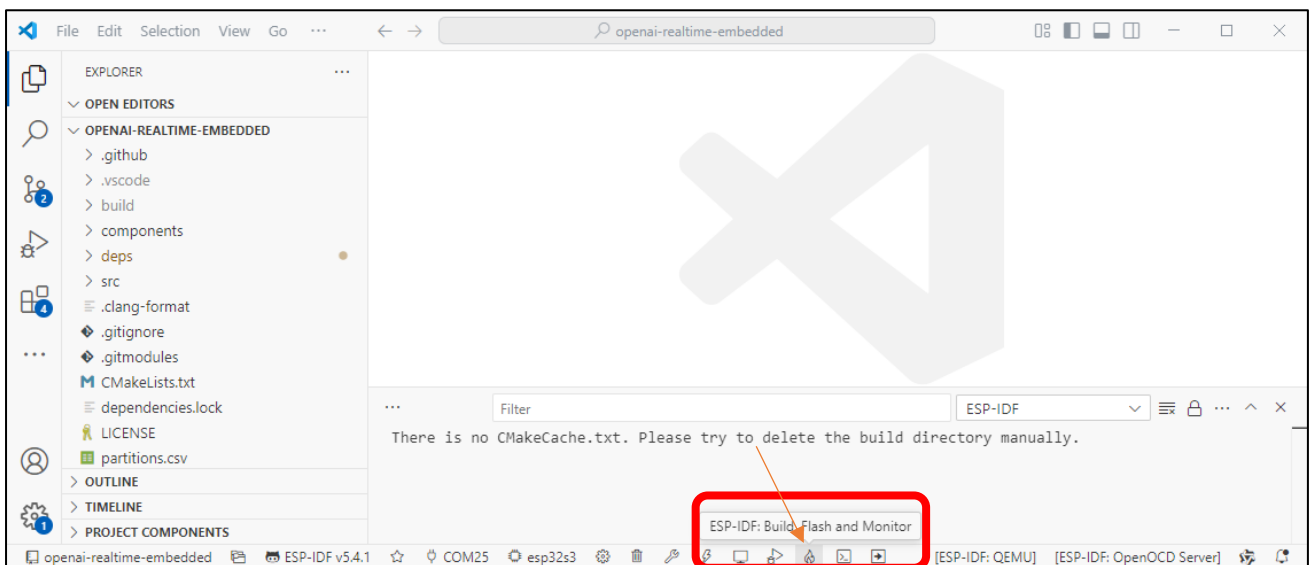
Now you have your own working OpenAI API keys. Feel free to explore more features on the website.

Code Compilation and Uploading

After ensuring everything is properly configured, let's start compiling the code.
Click "Full Clean" below to clear all previous compilation data.

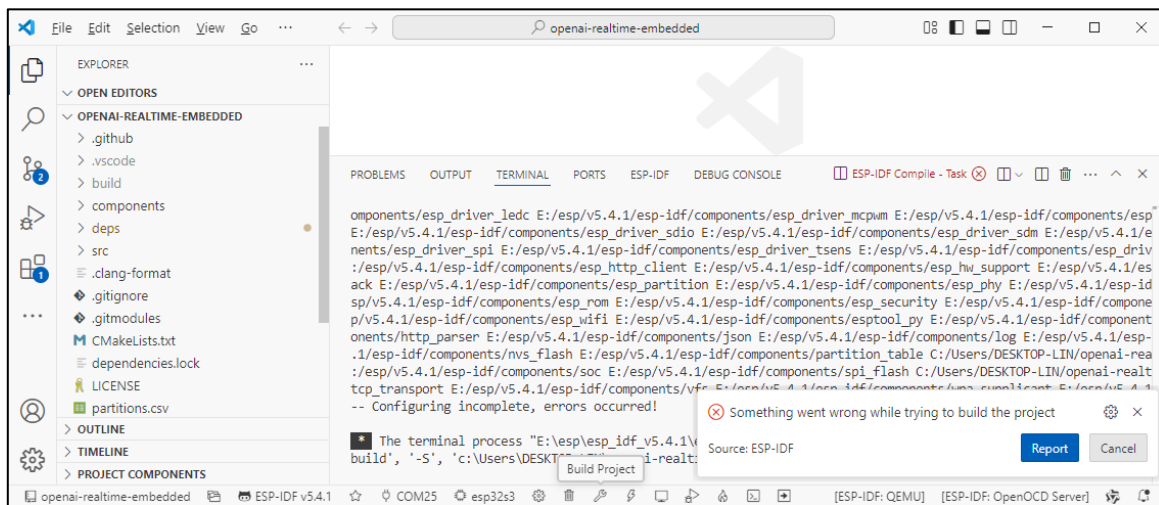


Click "ESP-IDF: Build, Flash and Monitor" below. It will compile the code, upload it to the ESP32-S3, and open the serial monitor.

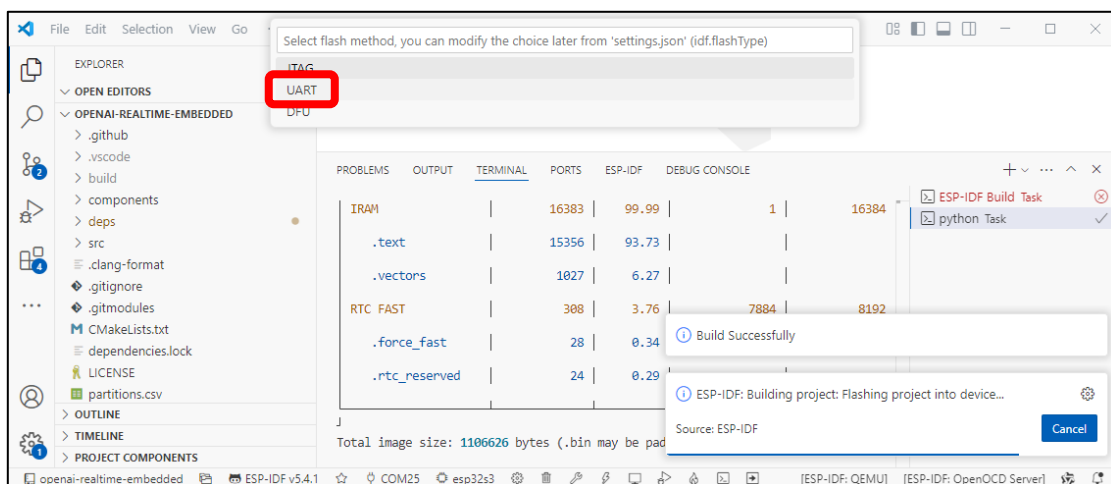


The first compilation may take longer—please ensure a stable internet connection and wait patiently.

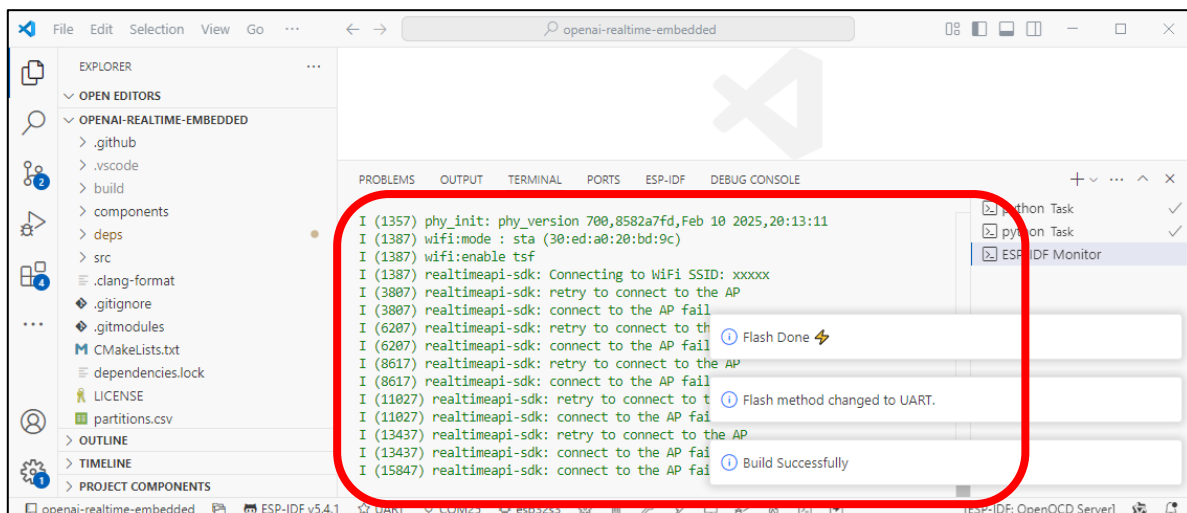
If you encounter the following error during compilation, it's typically caused by issues with the ESP-IDF toolchain. This often occurs due to network problems that prevented some toolchain components from installing properly. Please reinstall ESP-IDF. You can refer to [ESP-IDF V5.4.1](#).



When the code compilation is complete and you upload it for the first time, you will see the prompt below. Please select "UART".



After the upload completes, the serial monitor will automatically open and establish an internet connection to access OpenAI. You can now start chatting via the ESP32-S3.

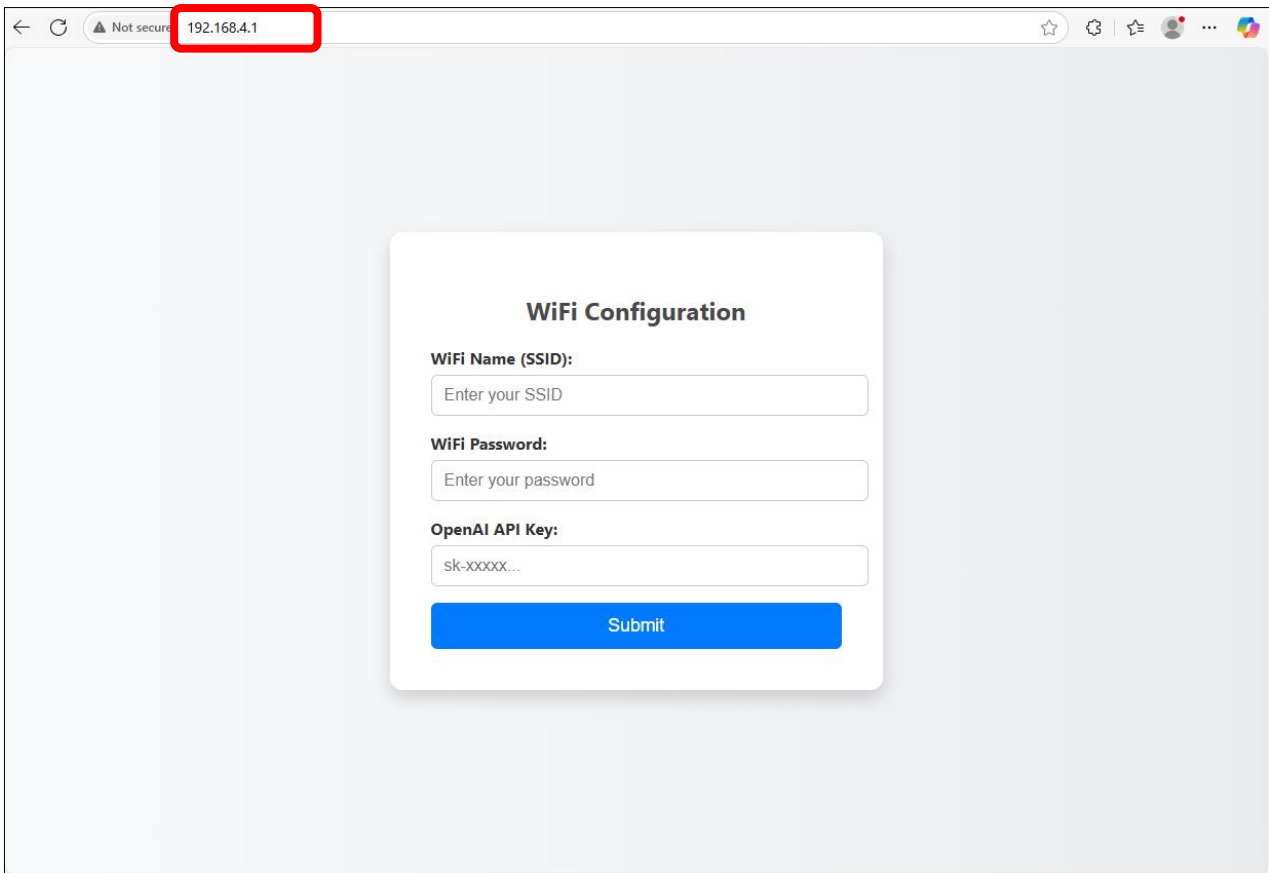


Computer End

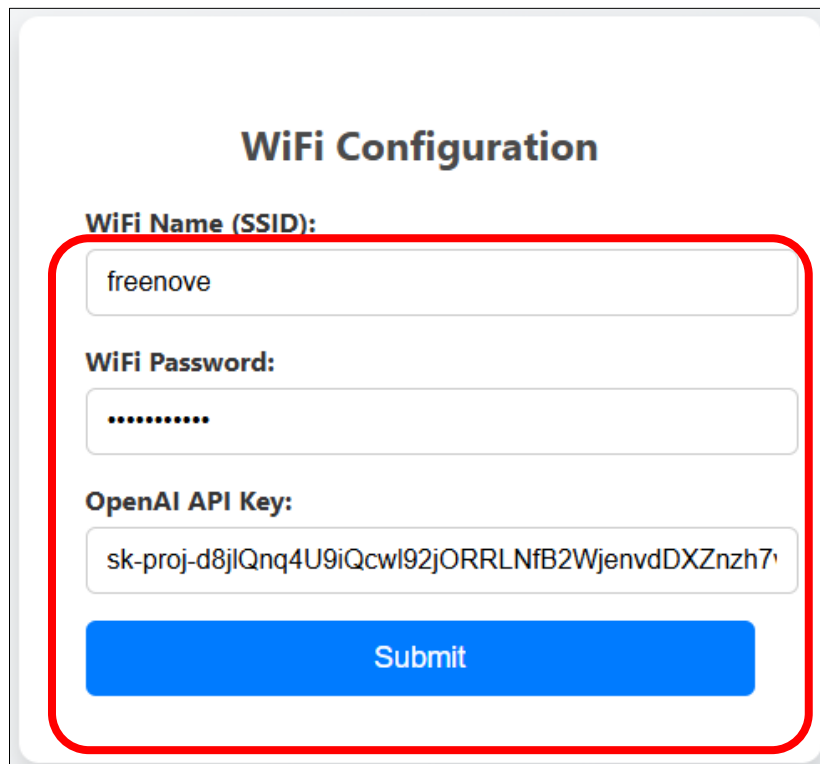
Enable Wi-Fi, locate "**OpenAI**" in the Wi-Fi settings, and click "**Connect**".



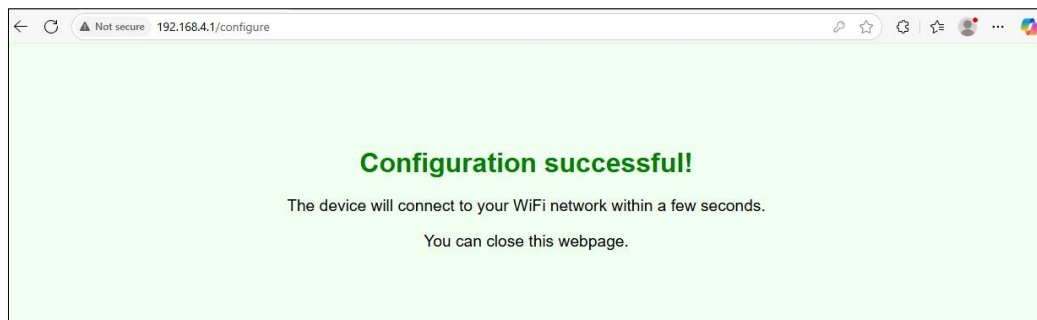
Open the browser, enter 192.168.4.1 in the address bar, and press Enter to access the network configuration page.



Enter the Wi-Fi name and password, as well as the API key of OpenAI, and then click Submit.



The following interface indicates successful configuration.



When you see the “**How can I help**” prompt, you can start communicate with openAI.

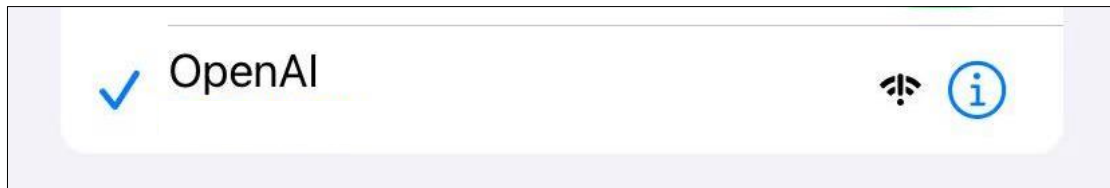


The chat logs will be displayed simultaneously in VS Code's Serial Monitor.

```
--- Waiting for the device to reconnect  
msg: How can I help?  
█
```

Mobile End

Enable Wi-Fi, locate "OpenAI" in the Wi-Fi settings to connect.



Open the browser on your phone, input **192.168.4.1** in the address bar to enter the network settings interface.

A screenshot of a mobile browser displaying a 'WiFi Configuration' form. The form has three input fields: 'WiFi Name (SSID):' with the placeholder 'Enter your SSID', 'WiFi Password:' with the placeholder 'Enter your password', and 'OpenAI API Key:' with the placeholder 'sk-xxxxx...'. Below these fields is a blue 'Submit' button. At the bottom of the screen, the browser's address bar shows '192.168.4.1' highlighted with a red rectangle.

Enter the Wi-Fi name and password, as well as the API key of OpenAI, and then click **Submit**.

A screenshot of the same 'WiFi Configuration' form, but now the input fields are filled. The 'WiFi Name (SSID):' field contains 'freenove', the 'WiFi Password:' field contains a series of dots, and the 'OpenAI API Key:' field contains 'sk-proj-d8jIQnq4U9iQcwI92jORRLNfB2Wjem'. The blue 'Submit' button is still at the bottom. The browser's address bar at the bottom shows '192.168.4.1'. A red rectangle highlights the entire form area.

The following interface indicates successful configuration.



When you see the “**How can I help**” prompt, you can start communicate with openAI.



The chat logs will be displayed simultaneously in VS Code's Serial Monitor.

```
--- Waiting for the device to reconnect  
msg: How can I help?  
█
```